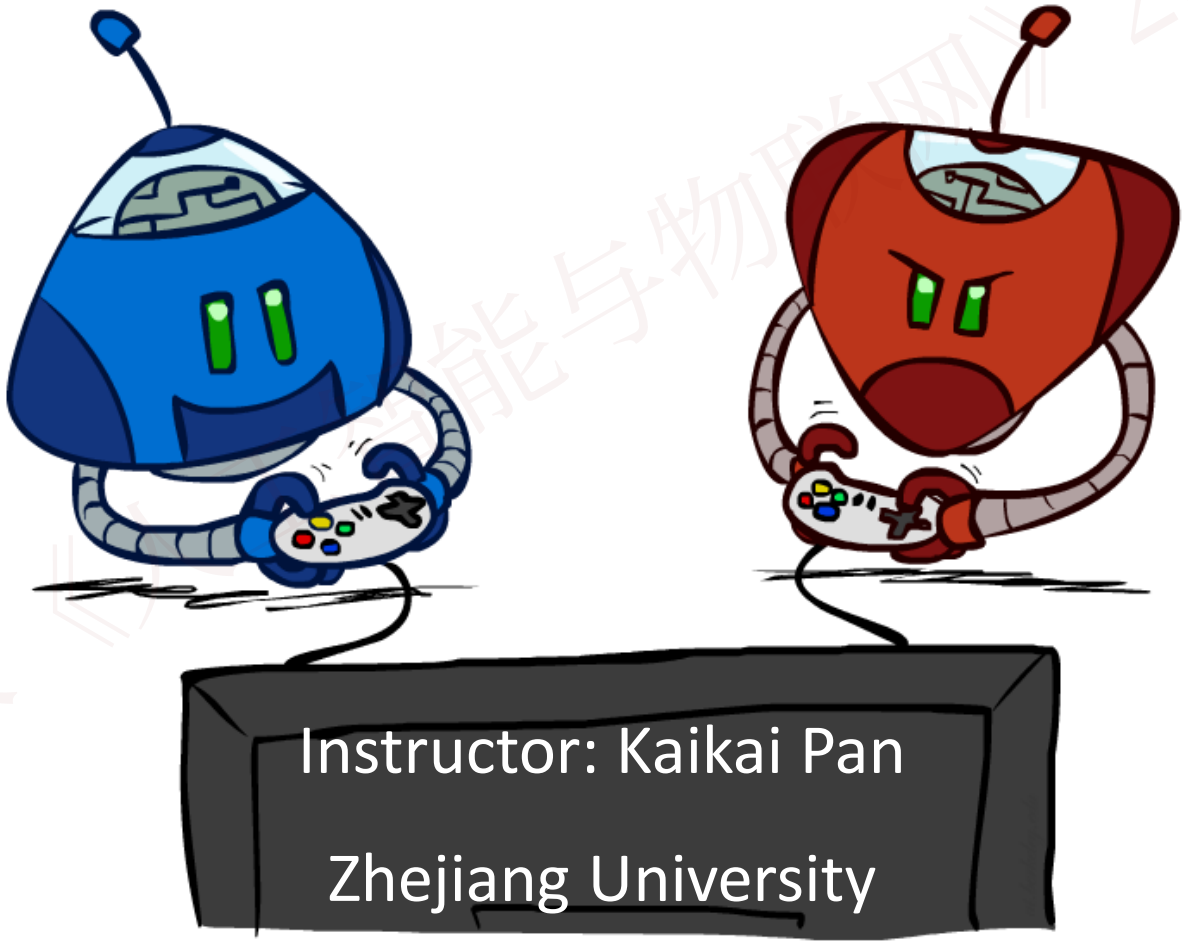
Artificial Intelligence & IoT

Adversarial Search



Instructor: Kaikai Pan

Zhejiang University

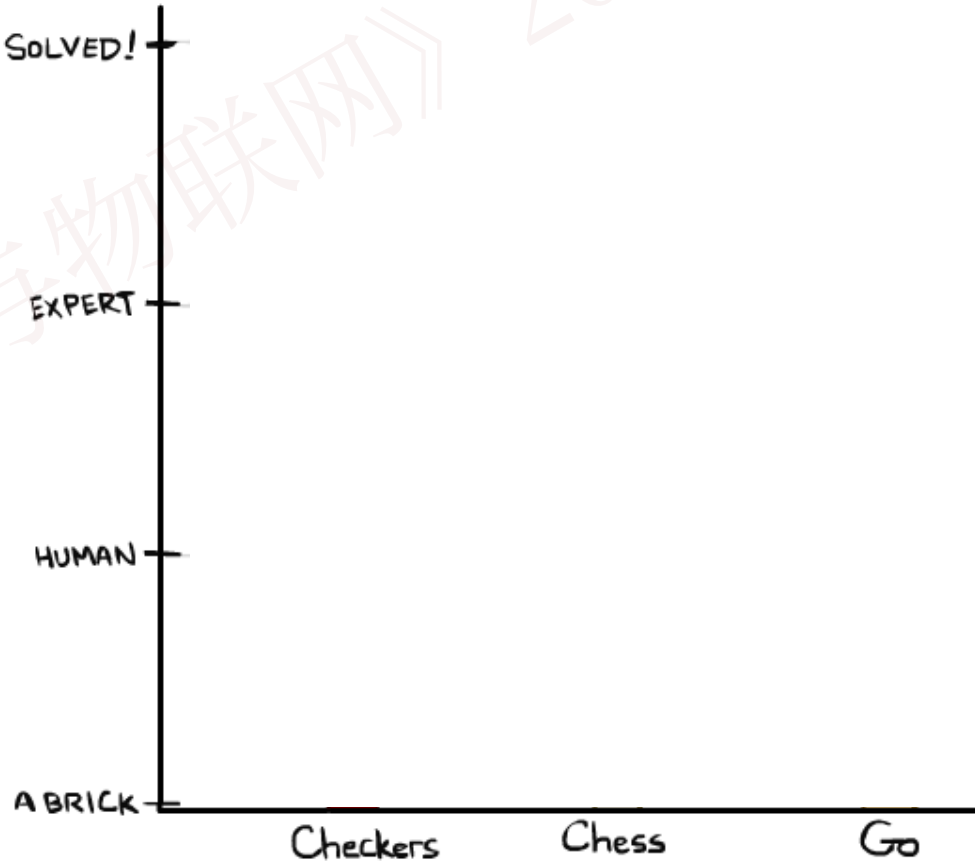
[These slides were adapted from the slides by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley. http://ai.berkeley.edu.]

Game Playing State-of-the-Art

- **Checkers:** 1950: First computer player. 1994: First computer champion: Chinook ended 40-year-reign of human champion Marion Tinsley using complete 8-piece endgame. 2007: Checkers solved!

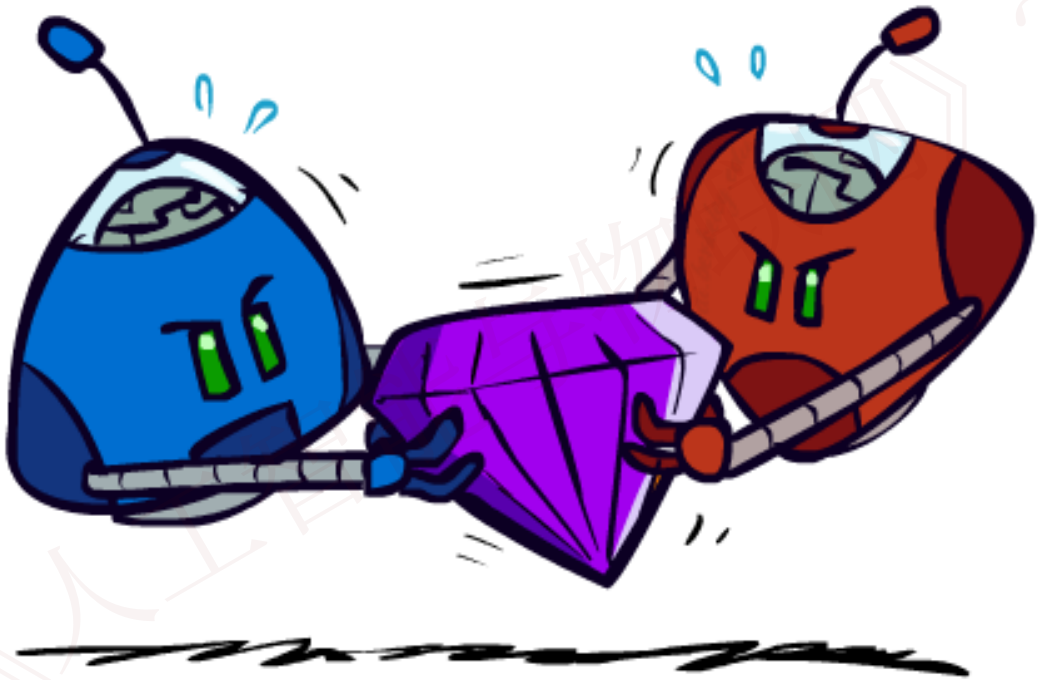
- **Chess:** 1997: Deep Blue defeats human champion Gary Kasparov. Deep Blue examined 200M positions per second, used very sophisticated evaluation and undisclosed methods for extending some lines of search up to 40 ply. Current programs are even better.

- **Go:** In go, b > 300! AlphaZero uses a novel form of reinforcement learning. AlphaZero plays games against itself, by combining a neural network with a powerful search algorithm. As it plays, the neural network is tuned and updated to predict moves, as well as the eventual winner of the games.



清华《人工智能与物联网》2026夏

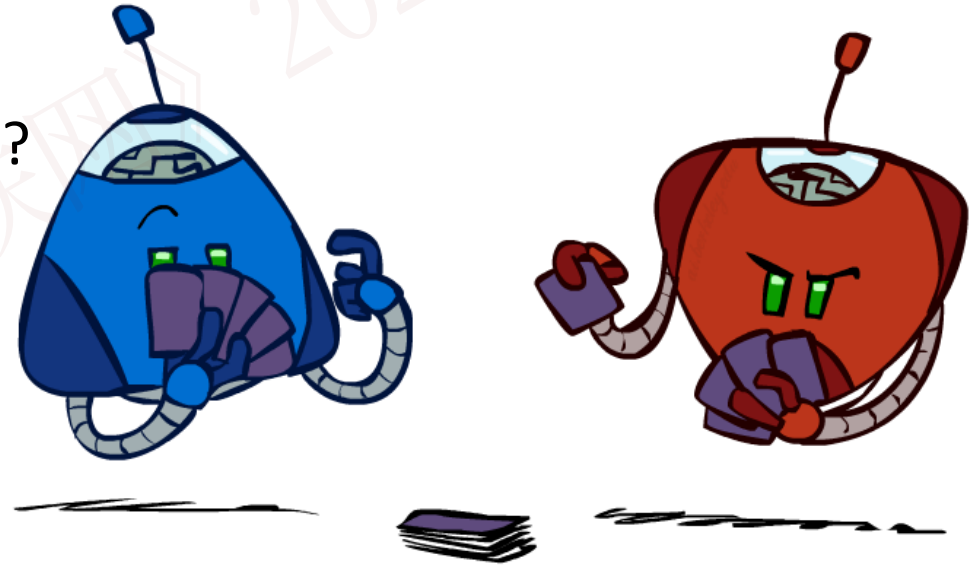
Adversarial Games



浙大《人工智能》 2026夏

Types of Games

- Many different kinds of games!
  - No-cooperative(非合作) or cooperative(合作式)
  - Deterministic (确定性博弈) or stochastic (随机博弈)?
  - One, two, or more players?
  - Zero-sum (零和博弈)?

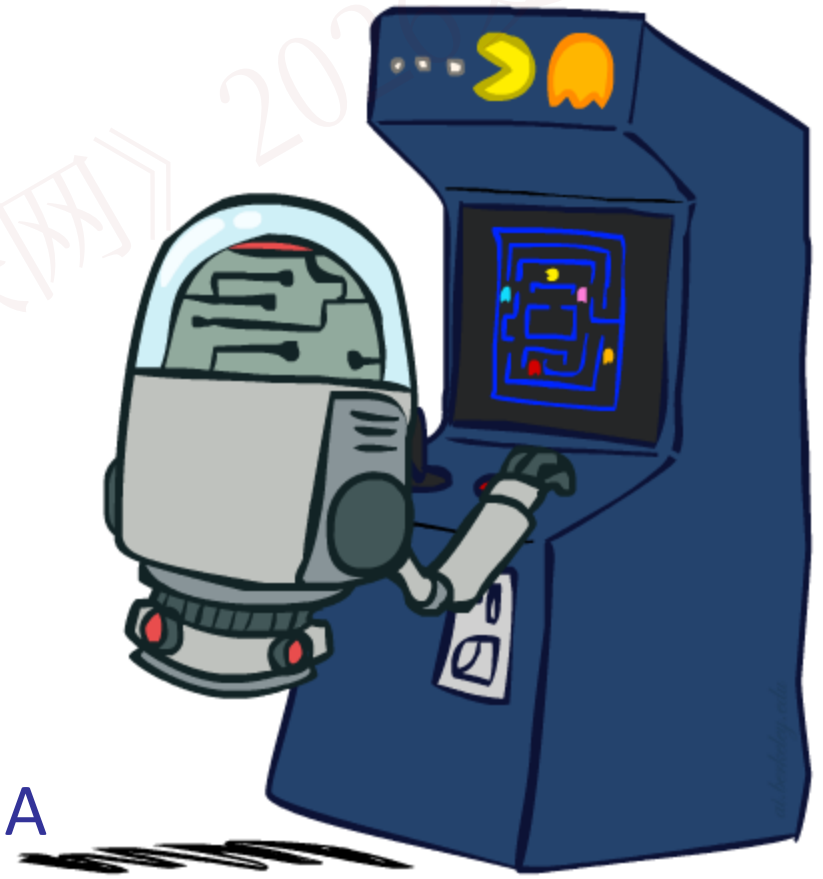


- Want algorithms for calculating a strategy (策略) which recommends an action from each state

浙大《人工智能与物联网》 2026夏

Deterministic Games

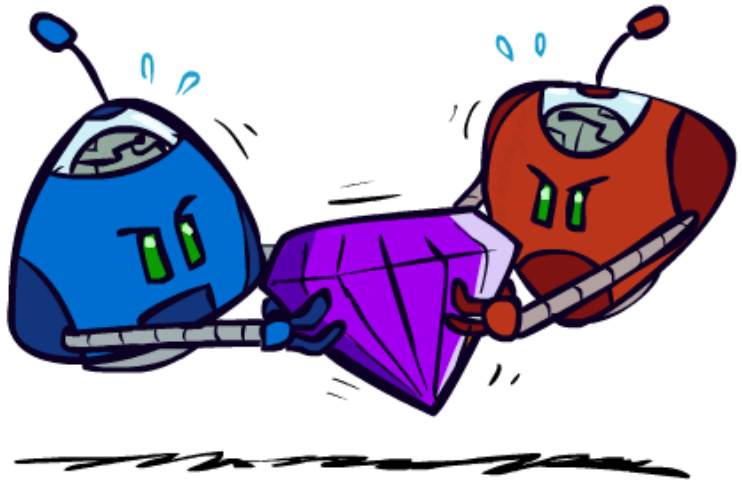
- Many possible formalizations, one is:
  - States: S (start at s_0)
  - Players (玩家): P={1...N} (usually take turns)
  - Actions: A (may depend on player / state)
  - Transition Function (转移函数): SxA → S
  - Terminal Test (终止测试): S → {t,f}
  - Utilities: SxP → R

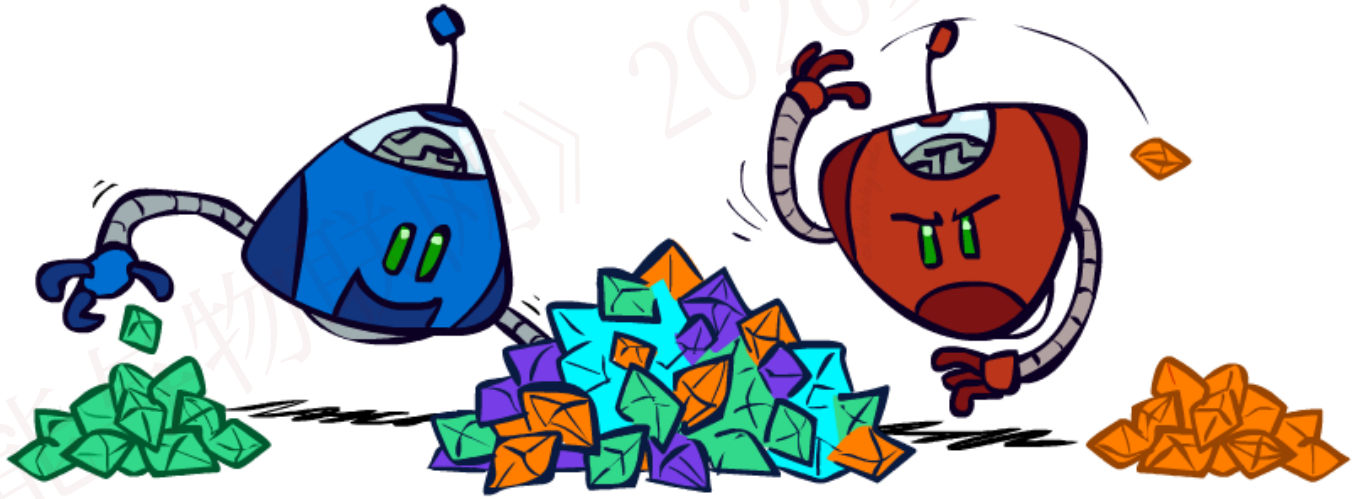


- Solution for a player is a strategy/policy: S → A

浙大《人工智能与物联网》2024春

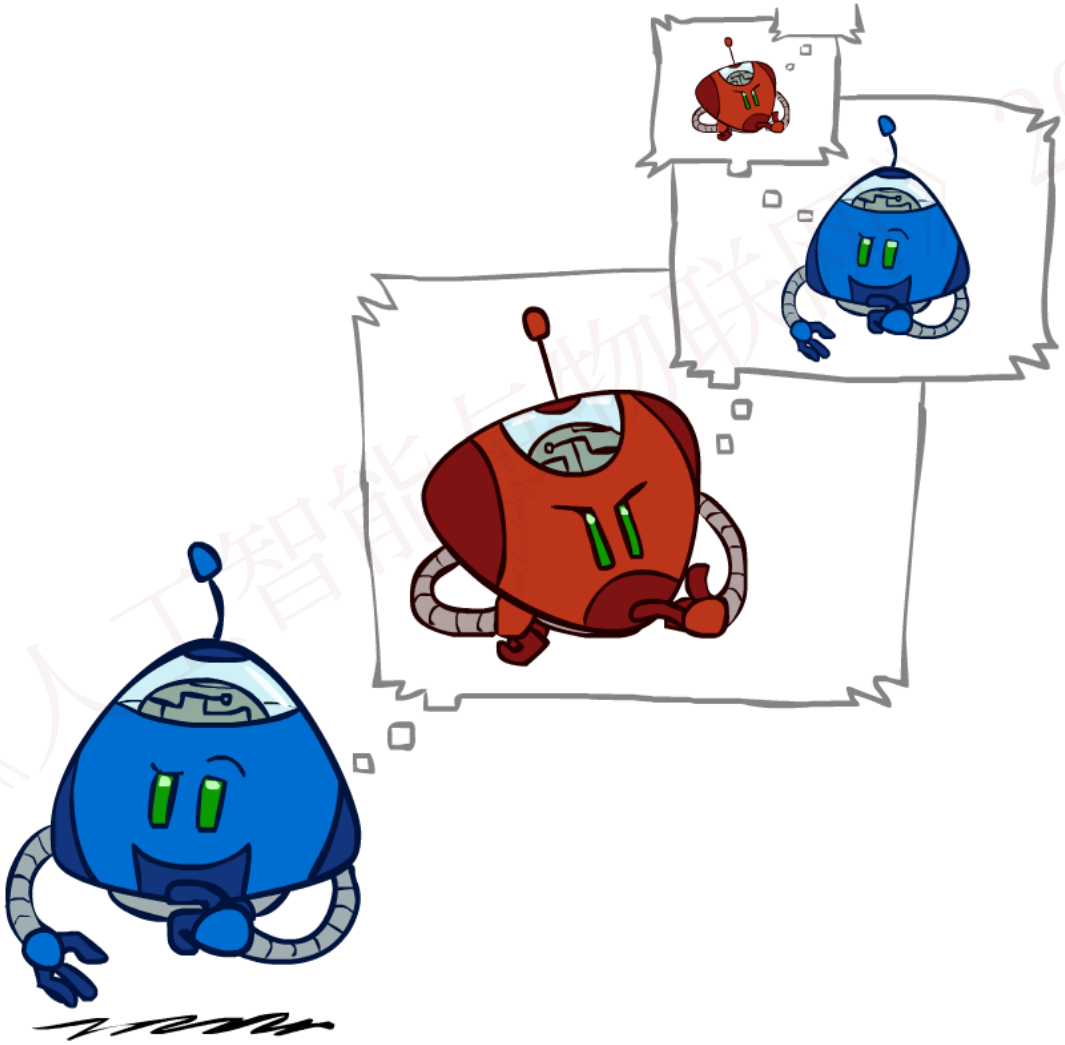
Zero-Sum Games





- Zero-Sum Games
  - Agents have opposite utilities (values on outcomes)
  - Let us think of a single value that one maximizes and the other minimizes
  - Adversarial, pure competition
- General Games (一般博弈)
  - Agents have independent utilities (values on outcomes)
  - Cooperation, indifference, competition, and more are all possible

Adversarial Search (对抗搜索)



浙大《人工智能与物联网》2026夏

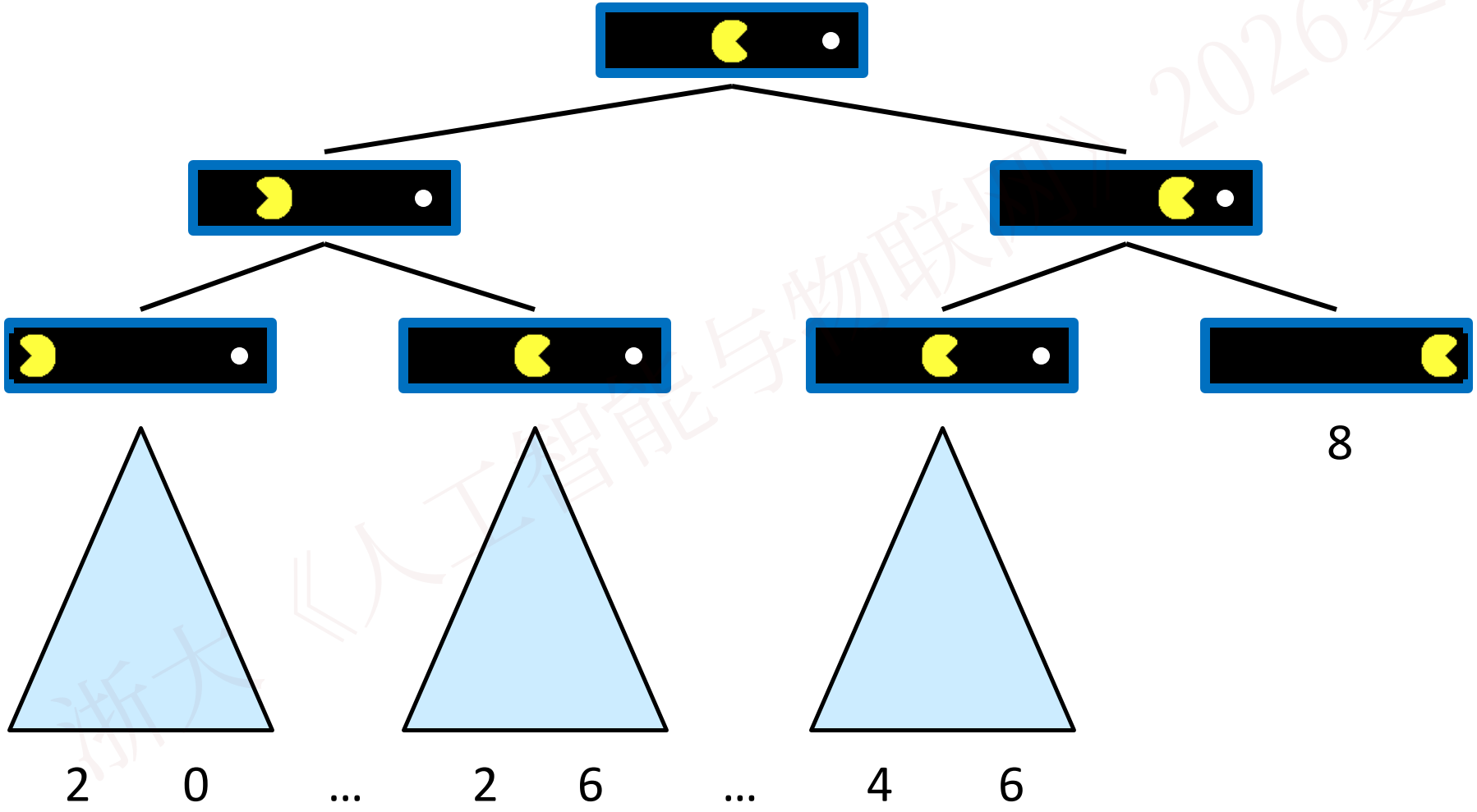
Adversarial Search Problem

- An adversarial game/search problem：
  - Two players: zero-sum. We focus on one of the player.
  - States: S (start at s_0)
  - Terminal State & Utility: S $\rightarrow$ {win:1, lose:-1}
  - Information: complete information & perfect information
  - Actions: A
  - Successor/Transaction Function: S x A $\rightarrow$ S (Deterministic)

- A solution for a player is a strategy (plan)
  - Best strategy: maximize the terminal utility.

浙大《人工智能与物联网》2026夏

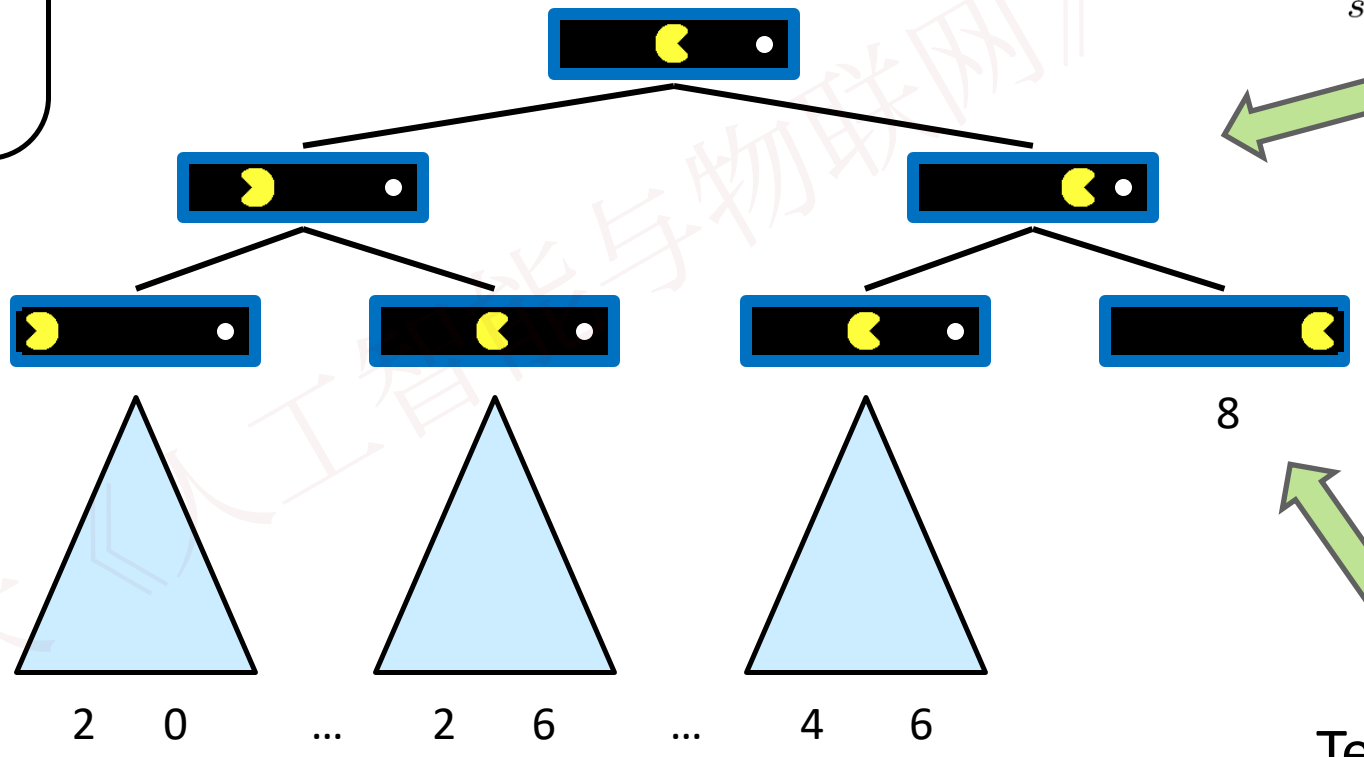
Single-Agent Trees



浙大《人工智能与物联网》2026夏

Value of a State

Value of a state:
The best achievable outcome (utility) from that state



Non-Terminal States:

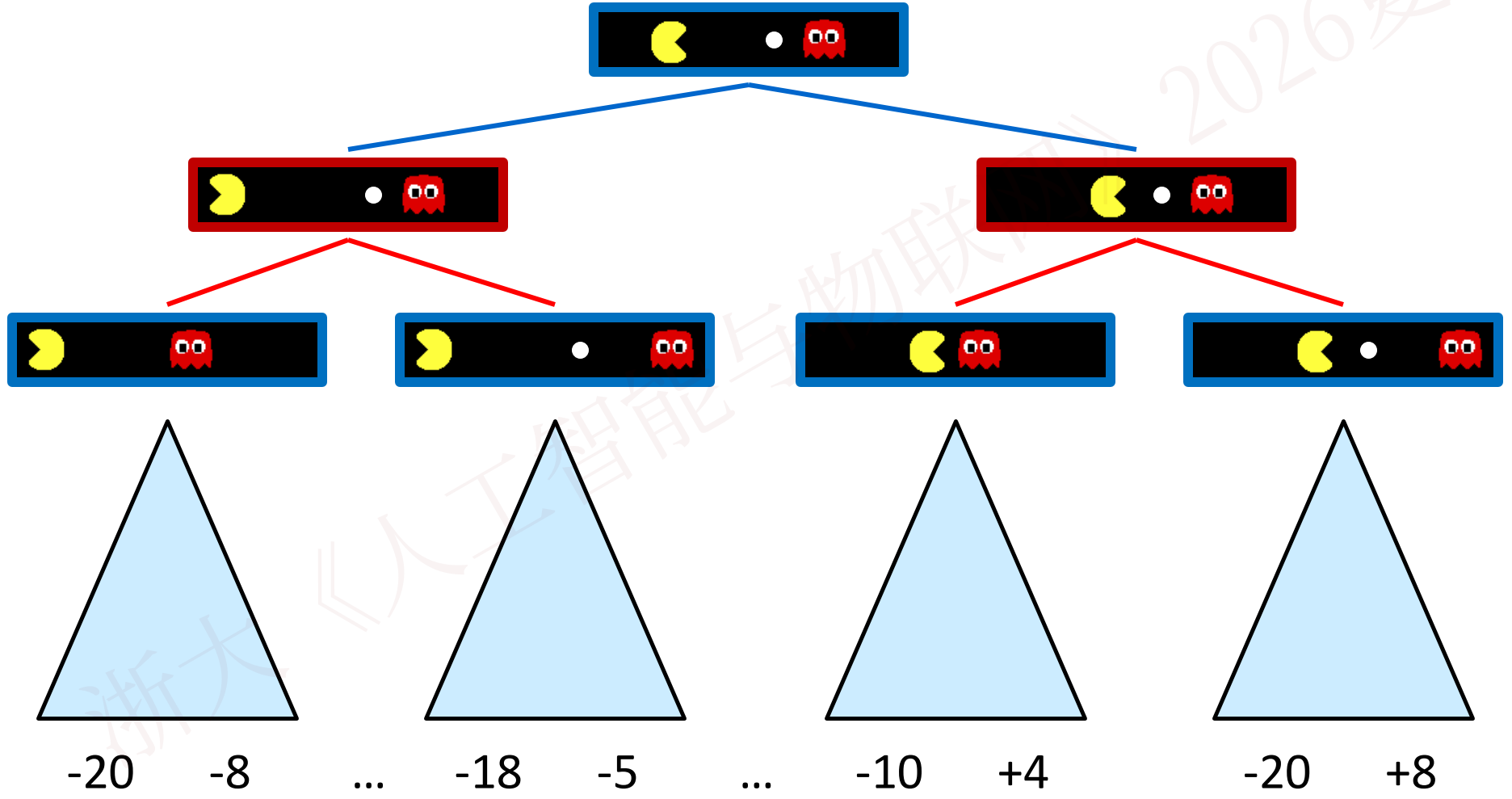
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Terminal States:

$$V(s) = \text{known}$$

浙大《人工智能与物联网》2026夏

Adversarial Game Trees



浙大《人工智能与物联网》2026夏

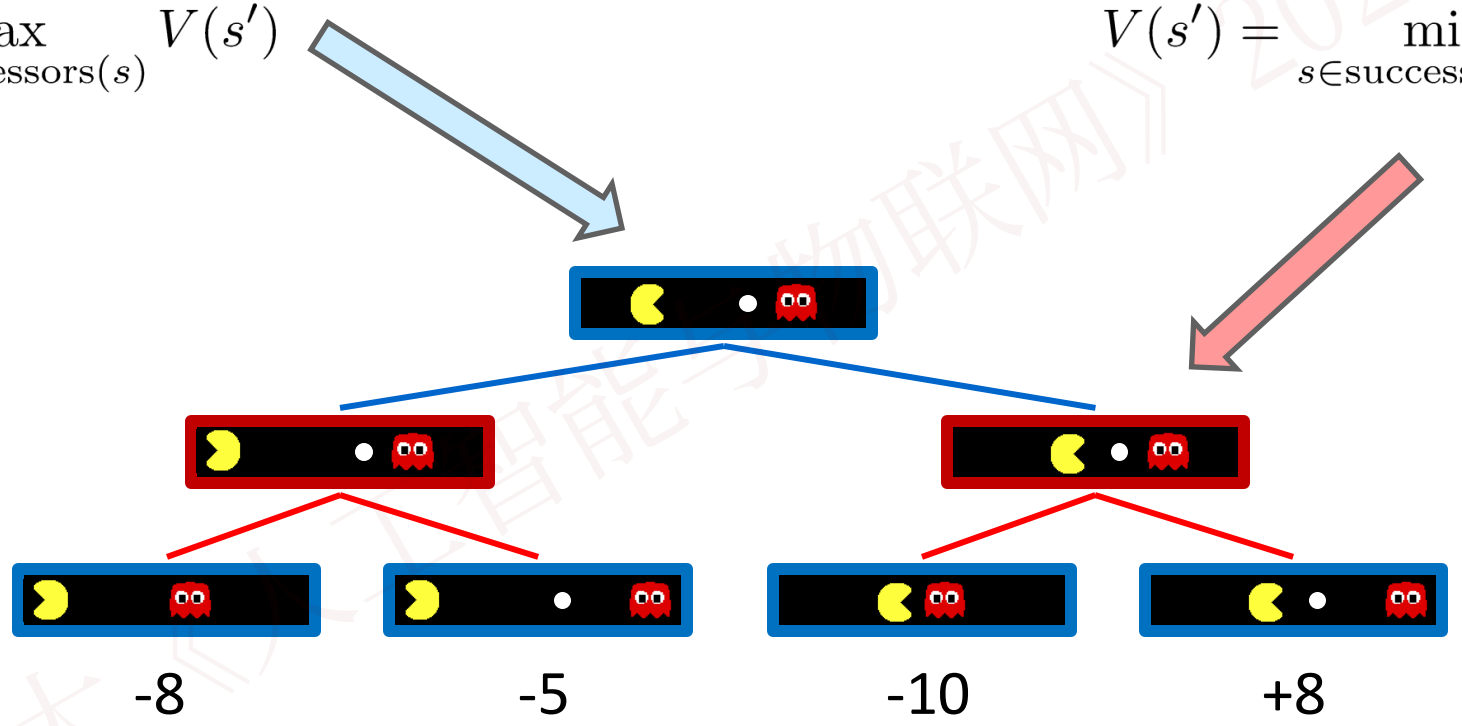
Minimax Values

States Under Agent's Control:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

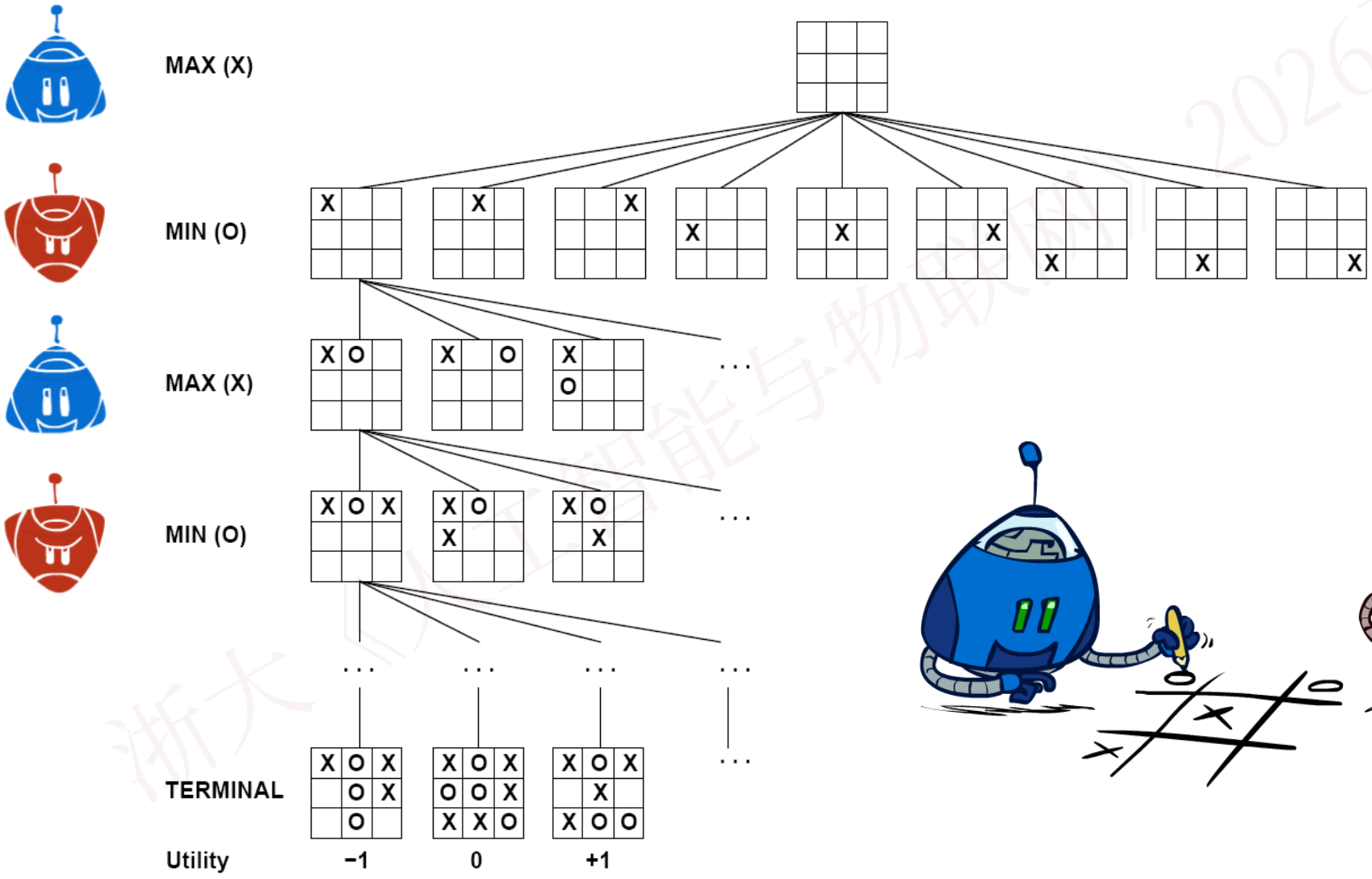


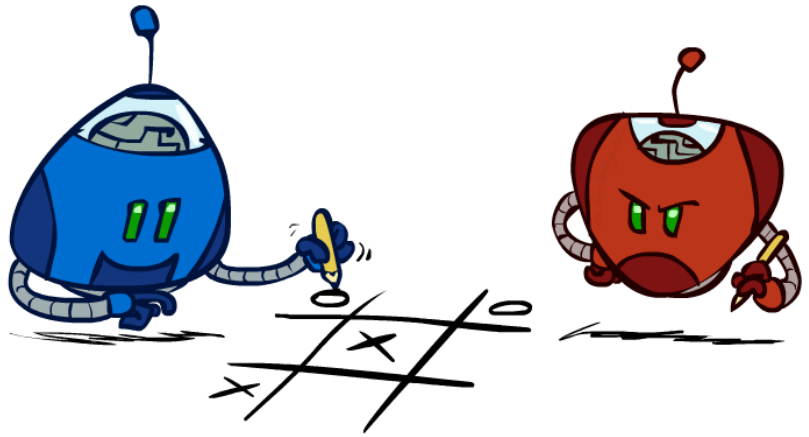
Terminal States:

$$V(s) = \text{known}$$

浙大人工智能与物联网 2026 夏

Tic-Tac-Toe Game Tree

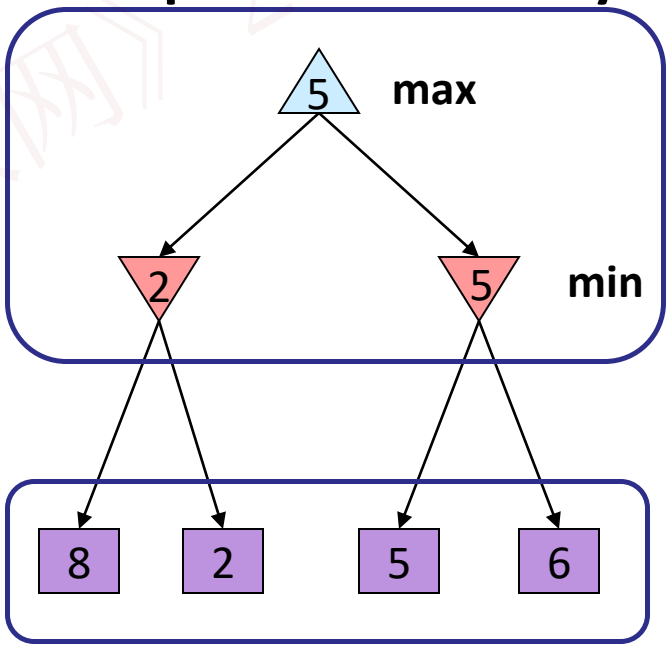




Adversarial Search (Minimax)

- Deterministic, zero-sum games:
  - Tic-tac-toe, chess, checkers
  - One player maximizes result
  - The other minimizes result
- Minimax search:
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary

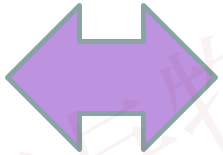
Minimax values: computed recursively



Terminal values: part of the game

Minimax Implementation

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```



```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

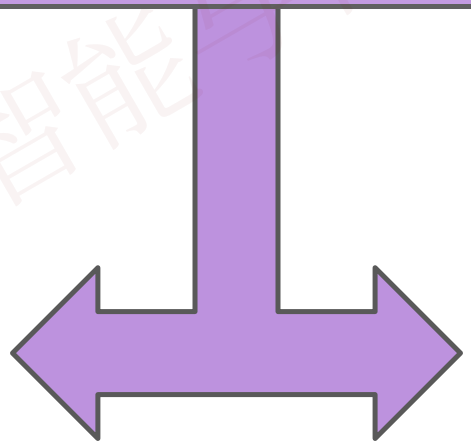
$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

浙大《人工智能与物联网》2026夏

Minimax Implementation (Dispatch)

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

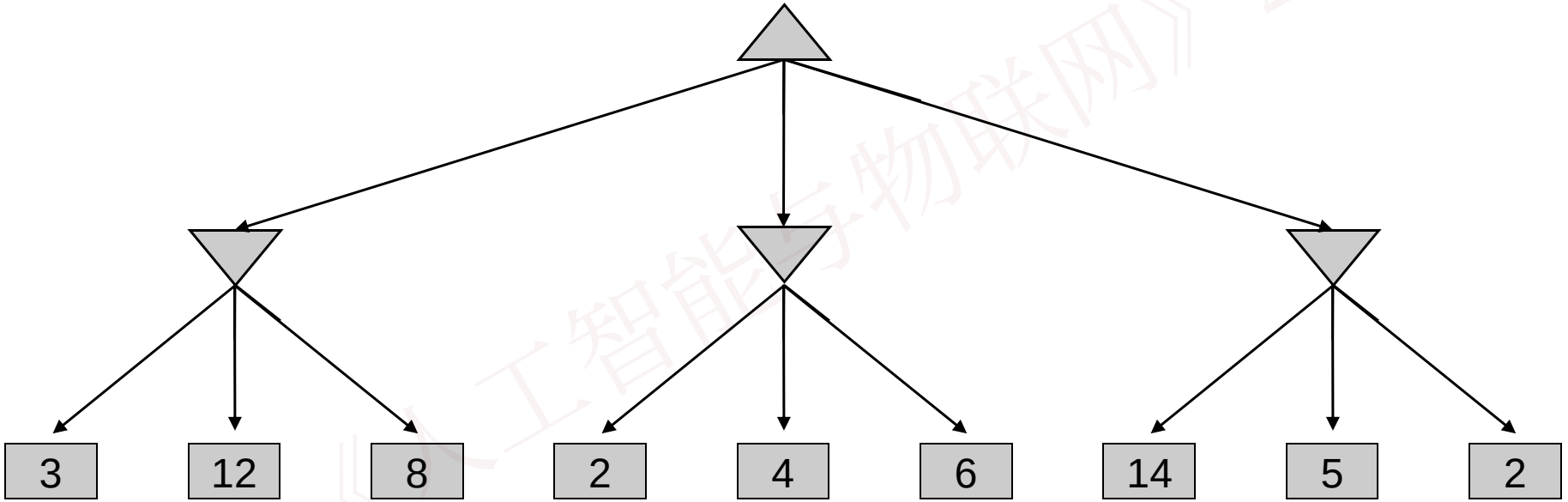


```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

浙大人工智能与物联网公 2026夏

Minimax Example

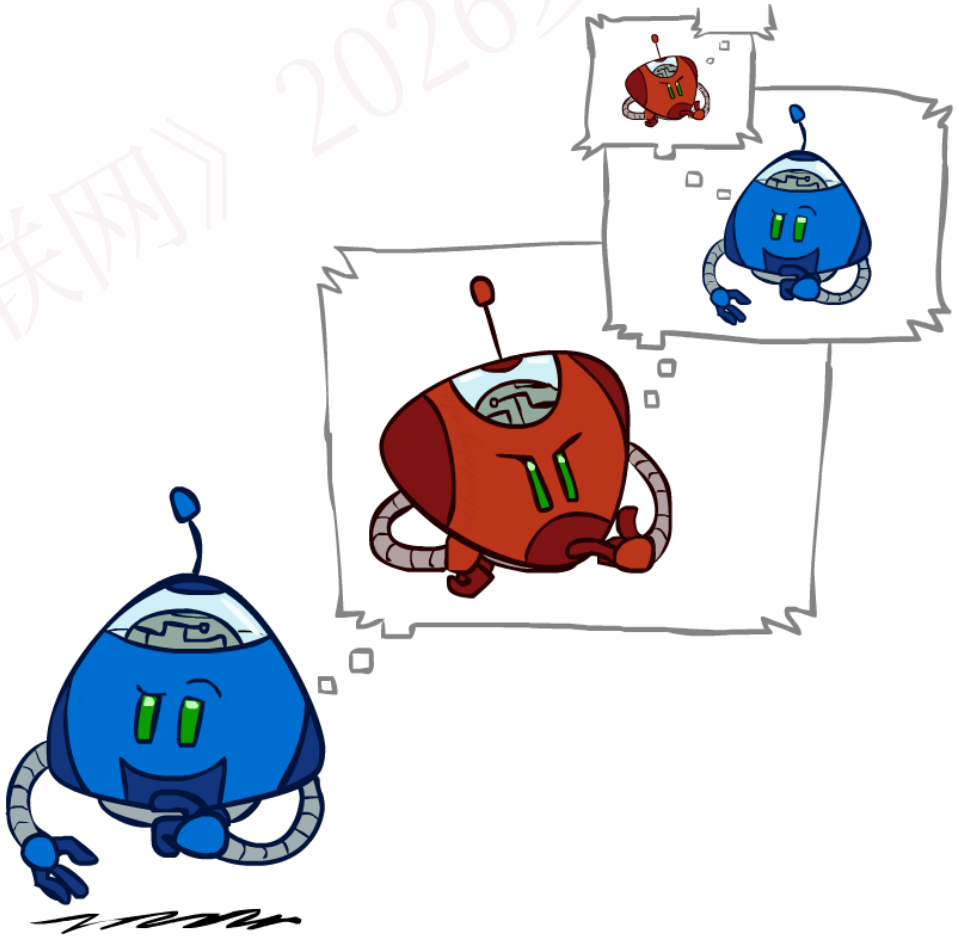


浙大·人工智能与物联网》2026夏

Minimax Efficiency

- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^m)$
  - Space: O(bm)

- Example: For chess, $b \approx 35$, $m \approx 100$
  - Exact solution is completely infeasible
  - But, do we need to explore the whole tree?



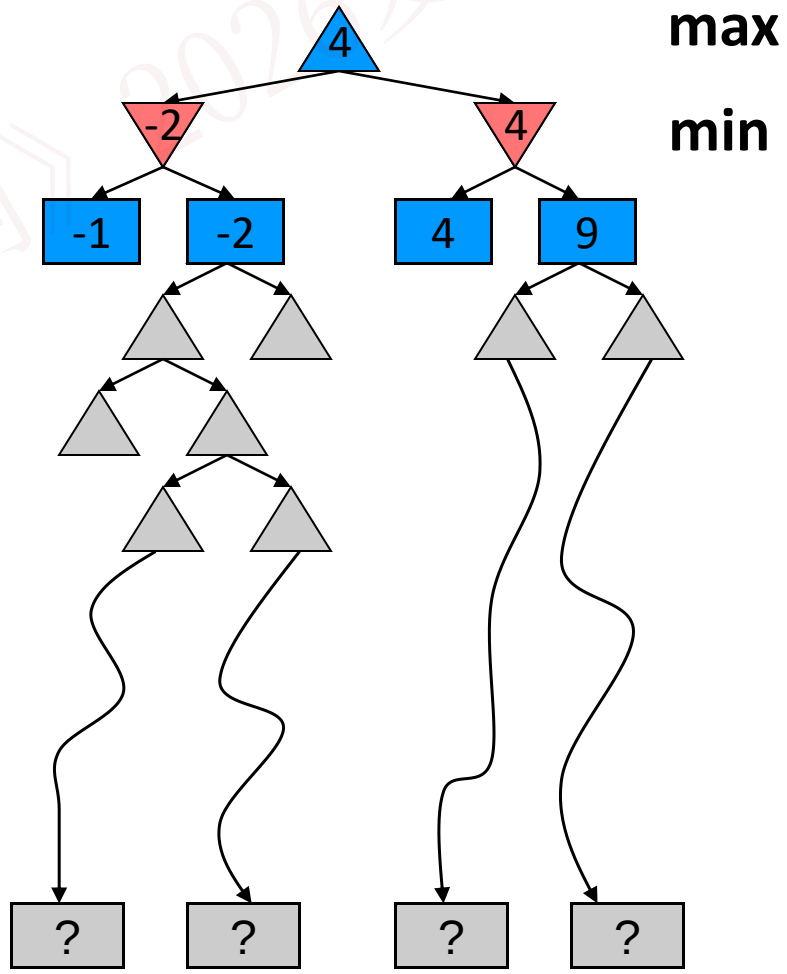
浙大《人工智能与物联网》2026夏

Resource Limits



Resource Limits

- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function for non-terminal positions
- Example:
  - Suppose we have 100 seconds, can explore 10K nodes / sec
  - So can check 1M nodes per action
  - α-β reaches about depth 8 – decent chess program
- Guarantee of optimal play is gone
- More plies (层) makes a BIG difference

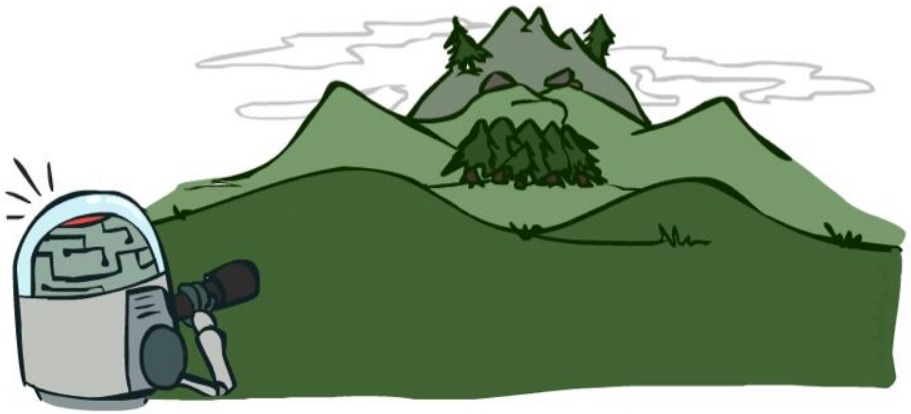


浙大人工智能与物联网公2026夏

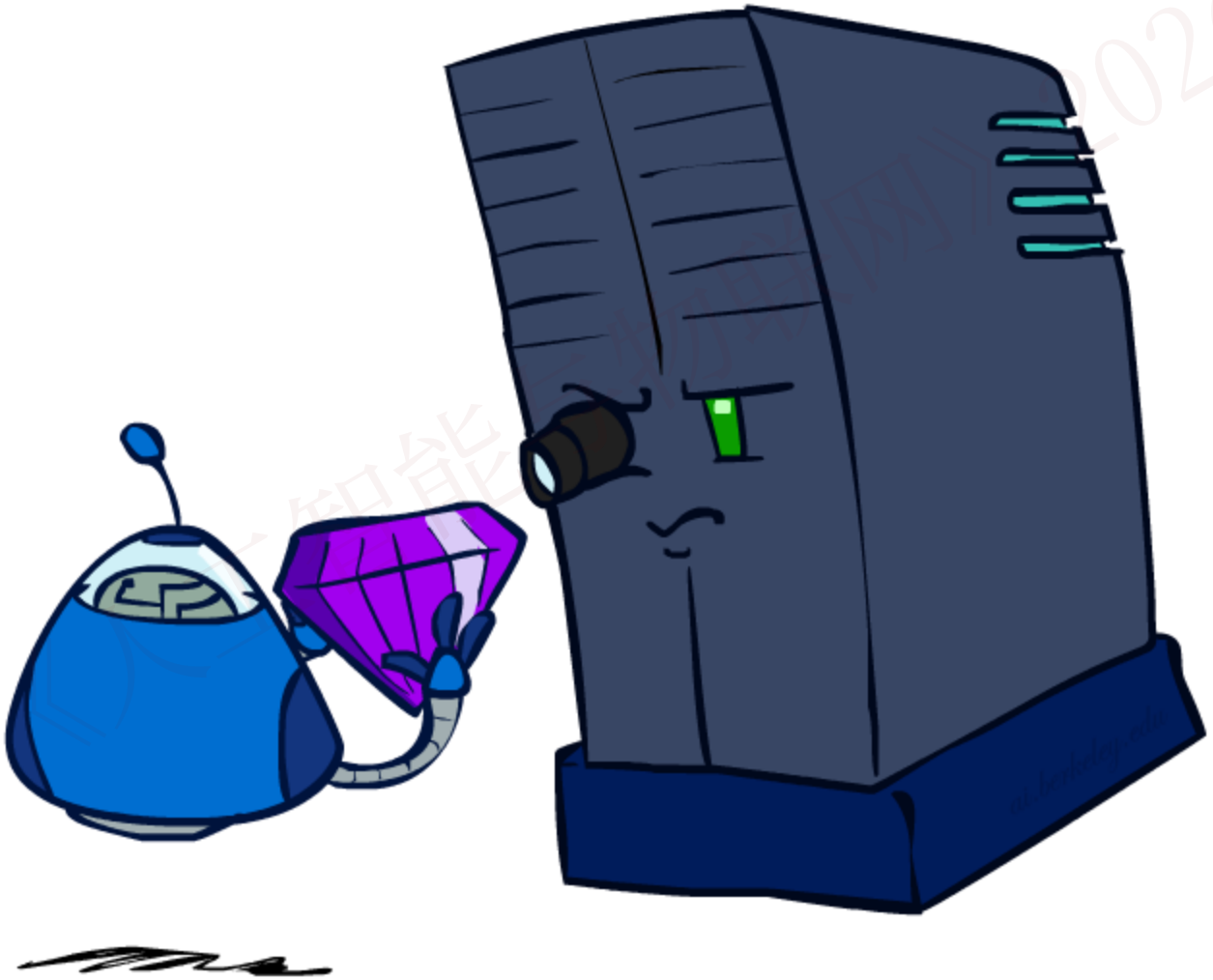
Depth Matters

- Evaluation functions are always imperfect
- The deeper in the tree, the easier and more accurate the evaluation function is， but the search is more complicated
- An important example of the tradeoff between complexity of features and complexity of computation



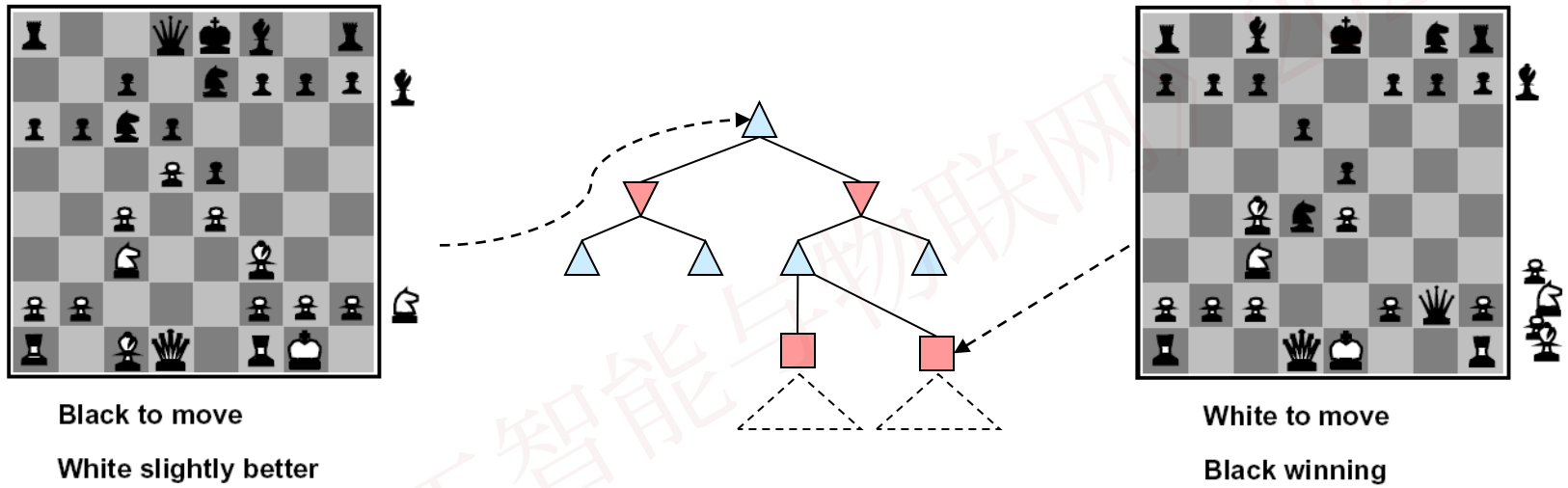


Evaluation Functions



Evaluation Functions

- Evaluation functions score non-terminal nodes in depth-limited search

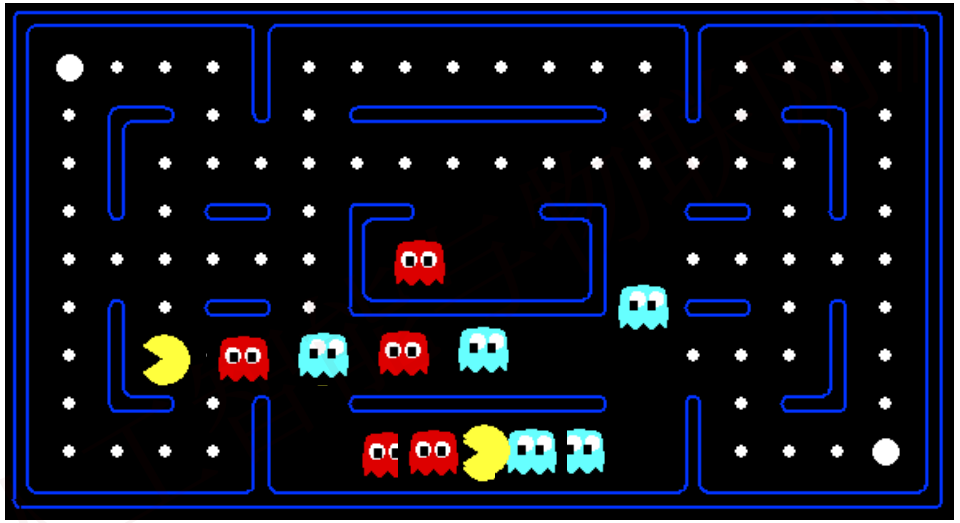


- Ideal function: returns the actual minimax value of the node
- In practice: typically weighted linear sum of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

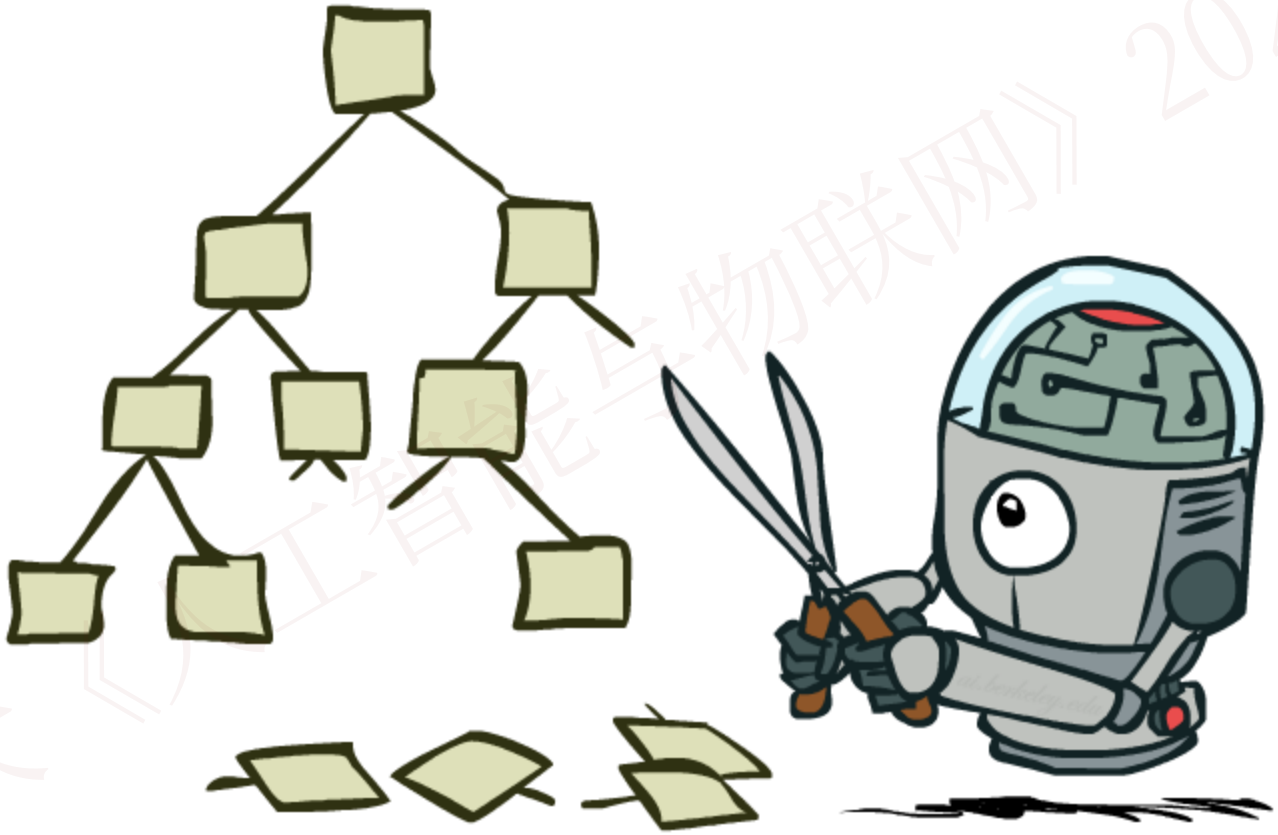
- e.g. $f_1(s)$ = (num white queens – num black queens), etc.

Evaluation for Pacman



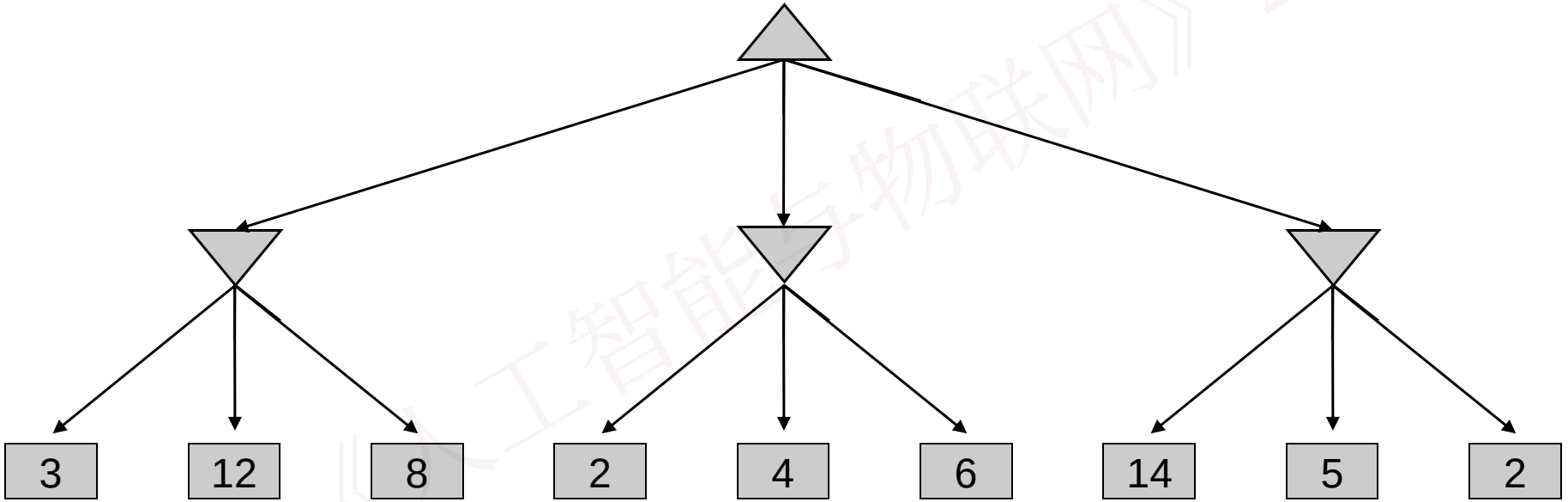
浙大《人 2026夏

Game Tree Pruning (剪枝)



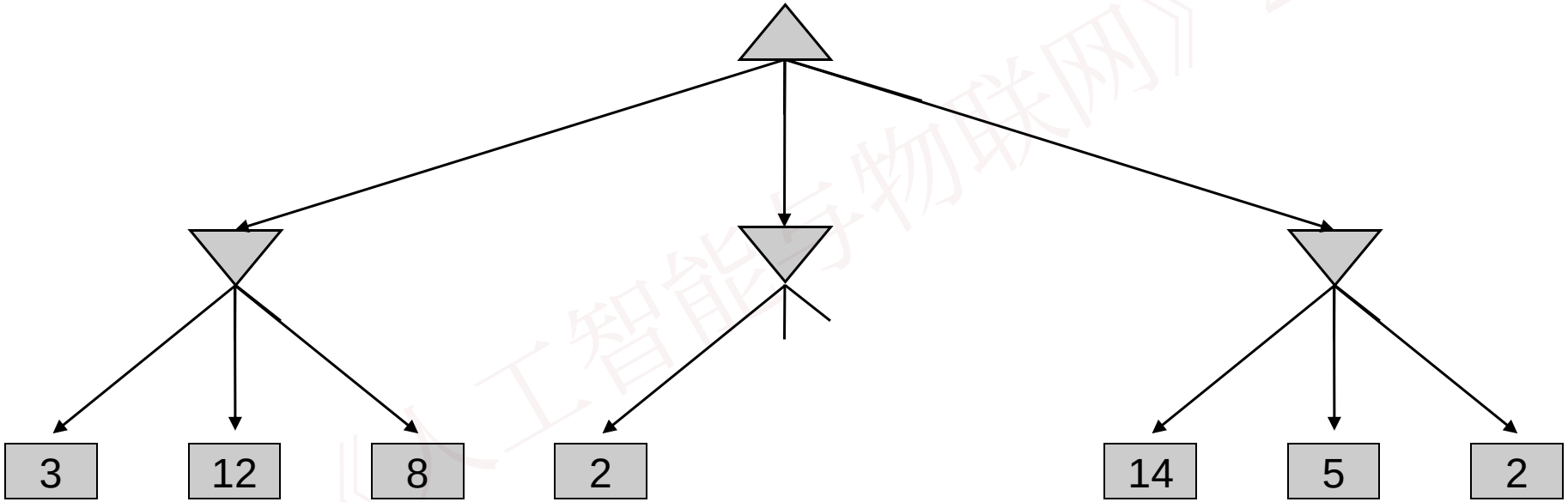
浙大《人工智能导论》2026夏

Minimax Example



浙大·人工智能与物联网》2026夏

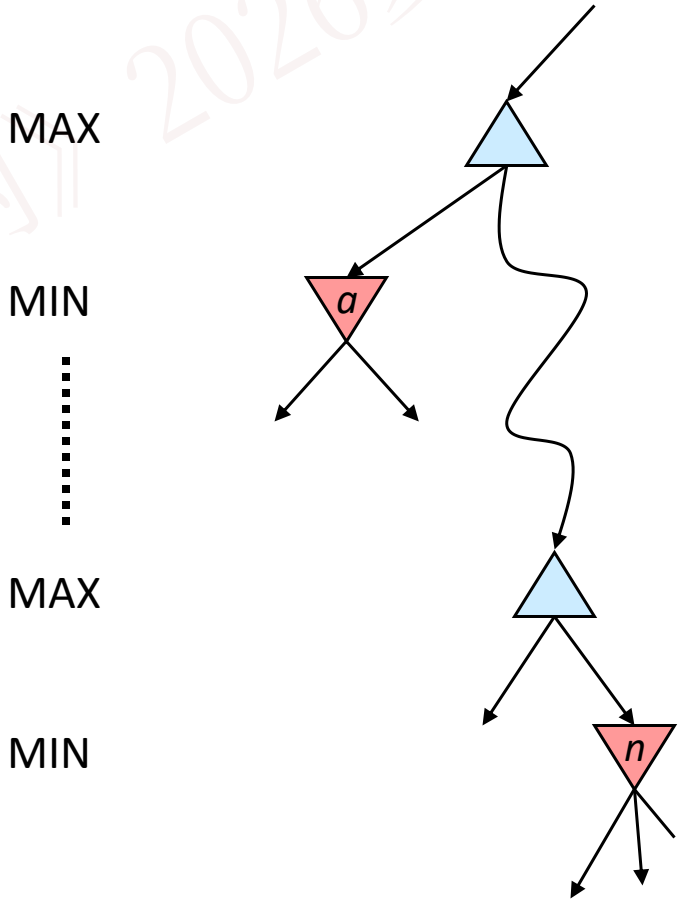
Minimax Pruning



浙大《人工智能与物联网》2026夏

Alpha-Beta Pruning

- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node *n*
  - We're looping over *n*'s children
  - *n*'s estimate of the childrens' min is dropping
  - Who cares about *n*'s value? MAX
  - Let *a* be the best value that MAX can get at any choice point along the current path from the root
  - If *n* becomes worse than *a*, MAX will avoid it, so we can stop considering *n*'s other children
- MAX version is symmetric



Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

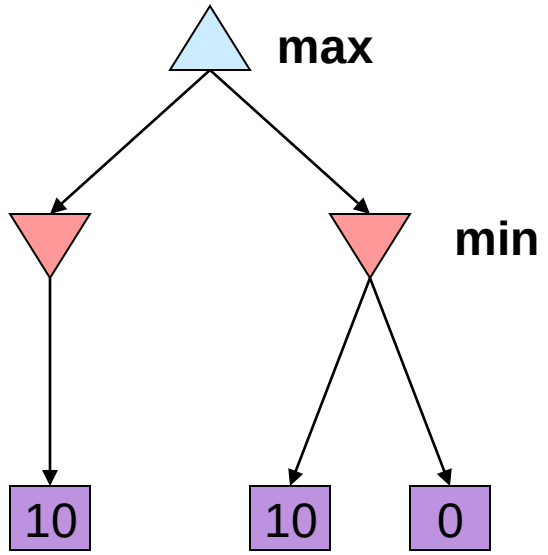
```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

浙大人工智能与物联网 2026夏

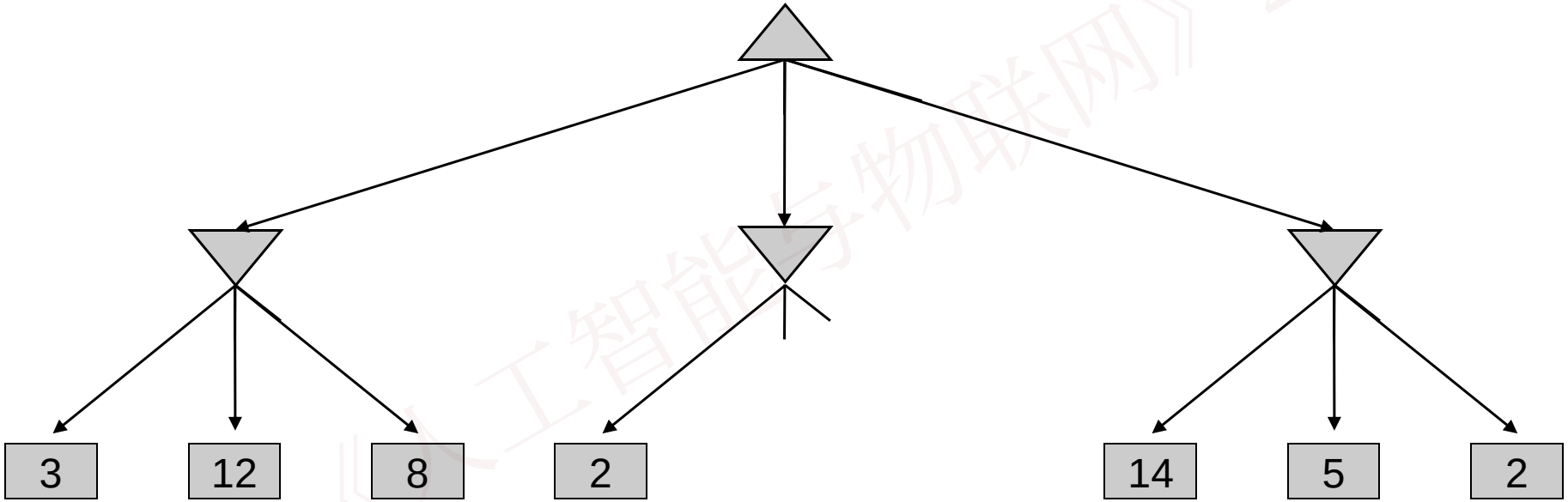
Alpha-Beta Pruning Properties

- This pruning has no effect on minimax value computed for the root!
- Good child ordering improves effectiveness of pruning
- With "perfect ordering":
  - Time complexity drops to $O(b^{m/2})$
  - Doubles solvable depth!
  - Full search of, e.g. chess, is still hopeless…



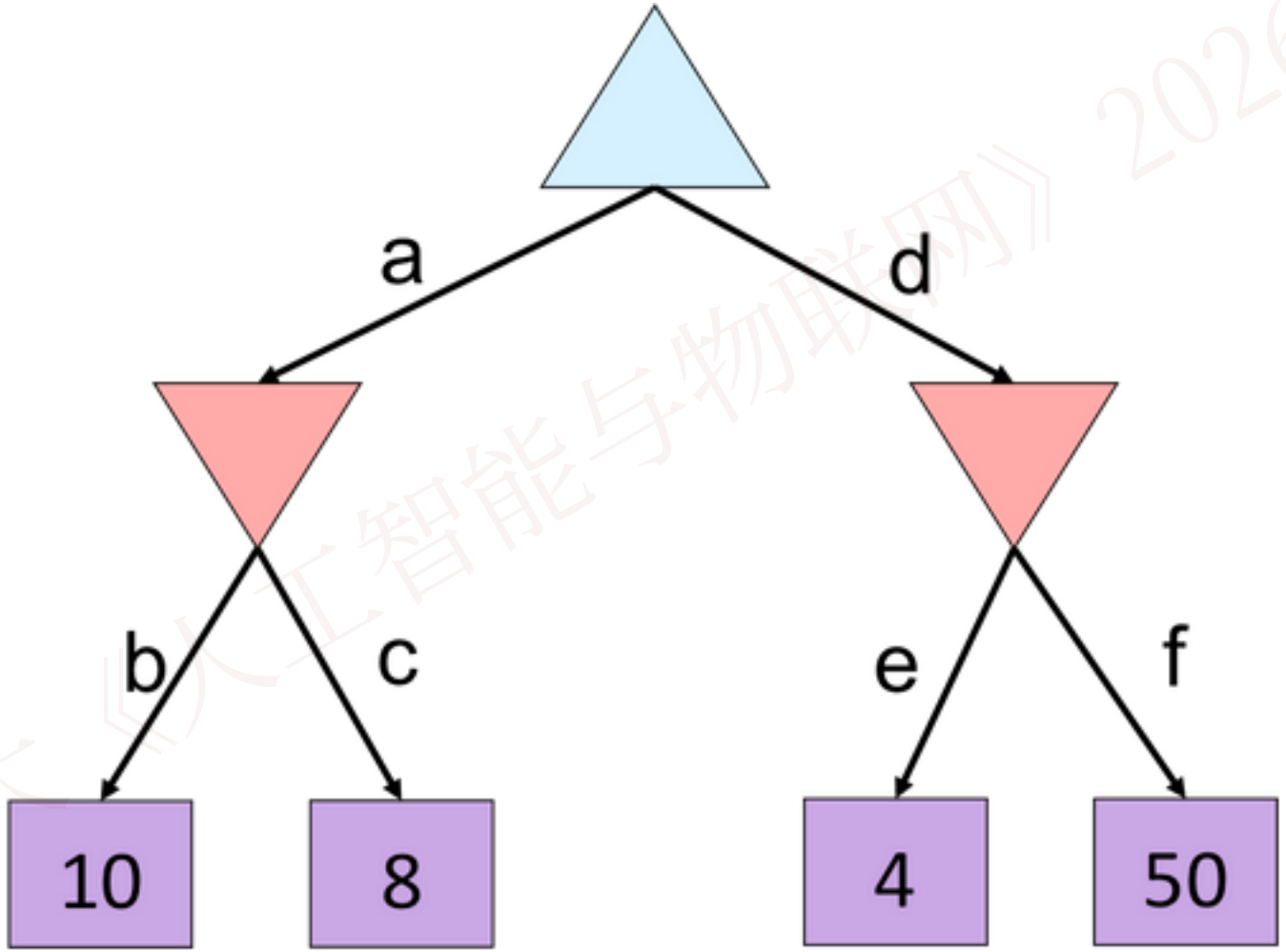
浙大《人工智能与物联网》2026夏

Minimax Pruning



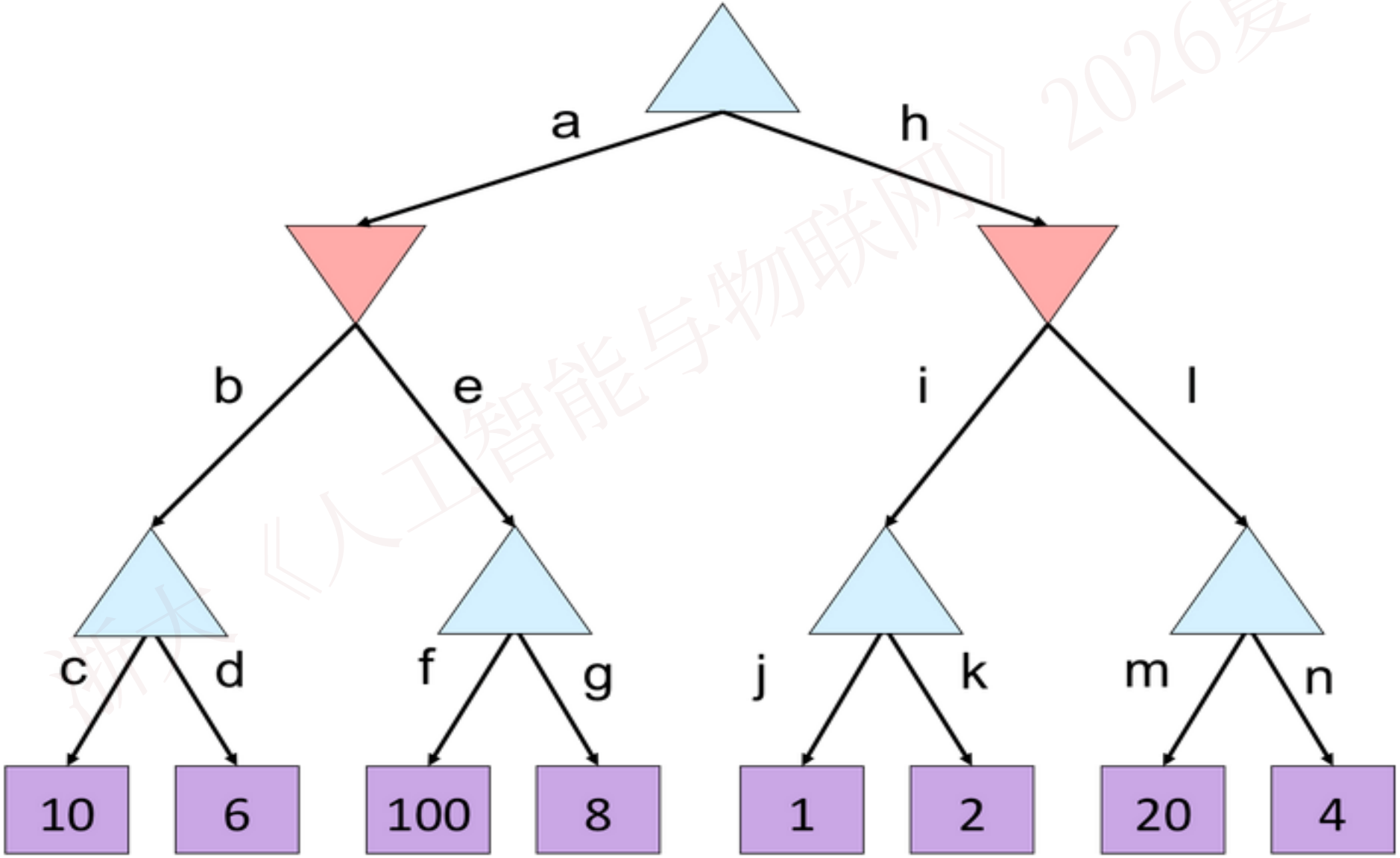
浙大《人工智能与物联网》2026夏

Alpha-Beta Quiz



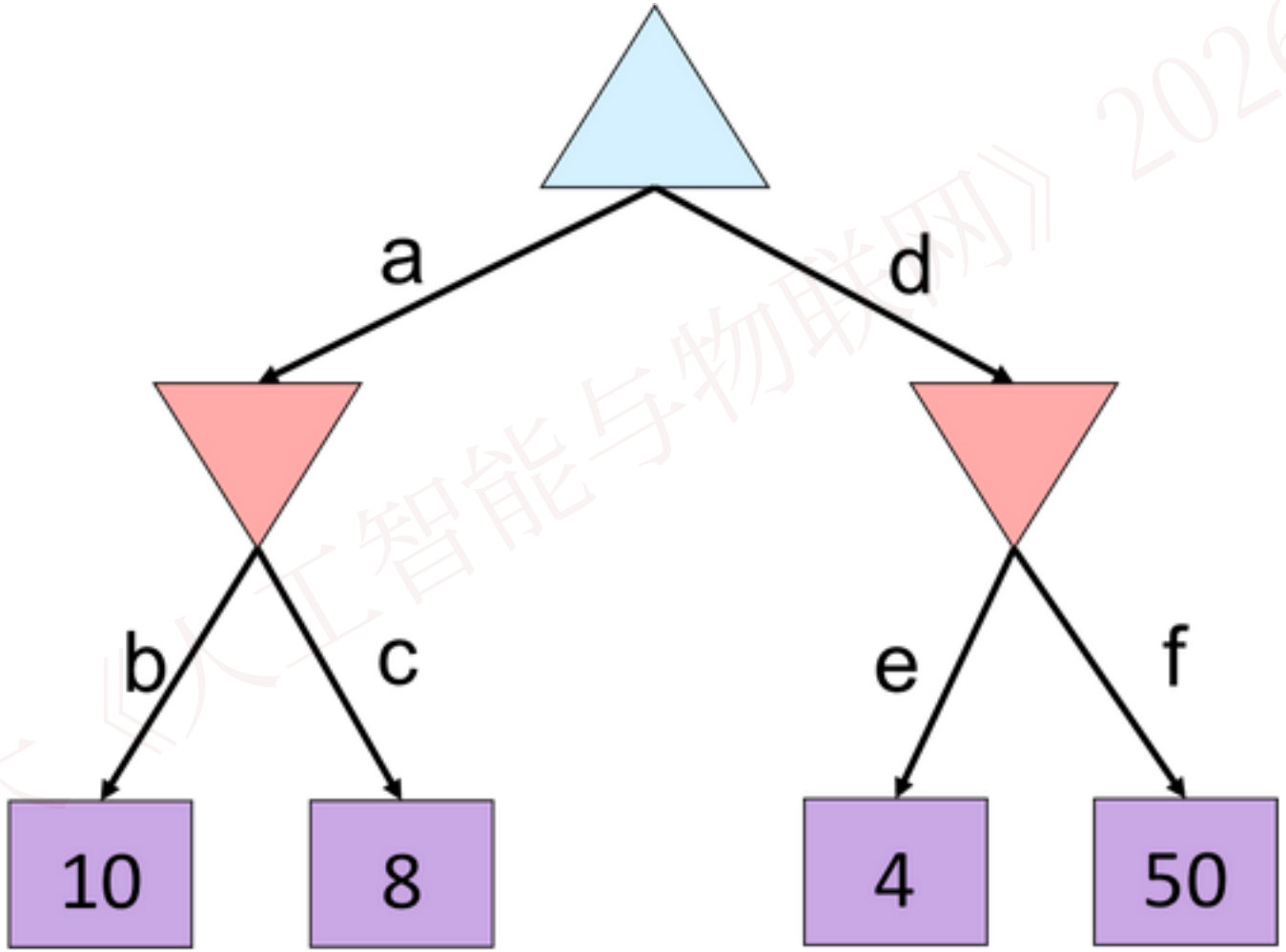
浙大·欧大汇智能与物联网》2026夏

Alpha-Beta Quiz 2



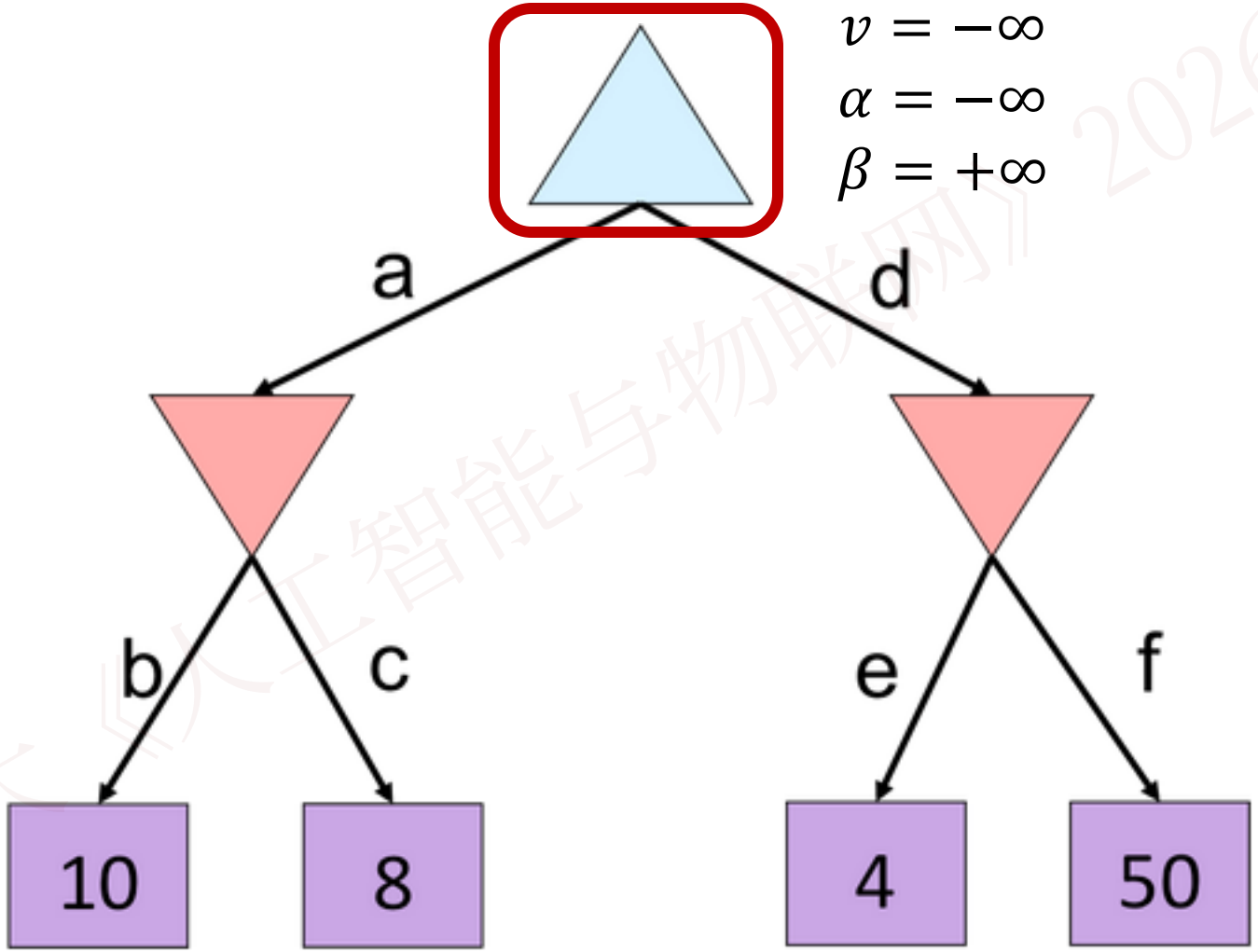
浙大《人工智能与物联网》 2026夏

Alpha-Beta Quiz



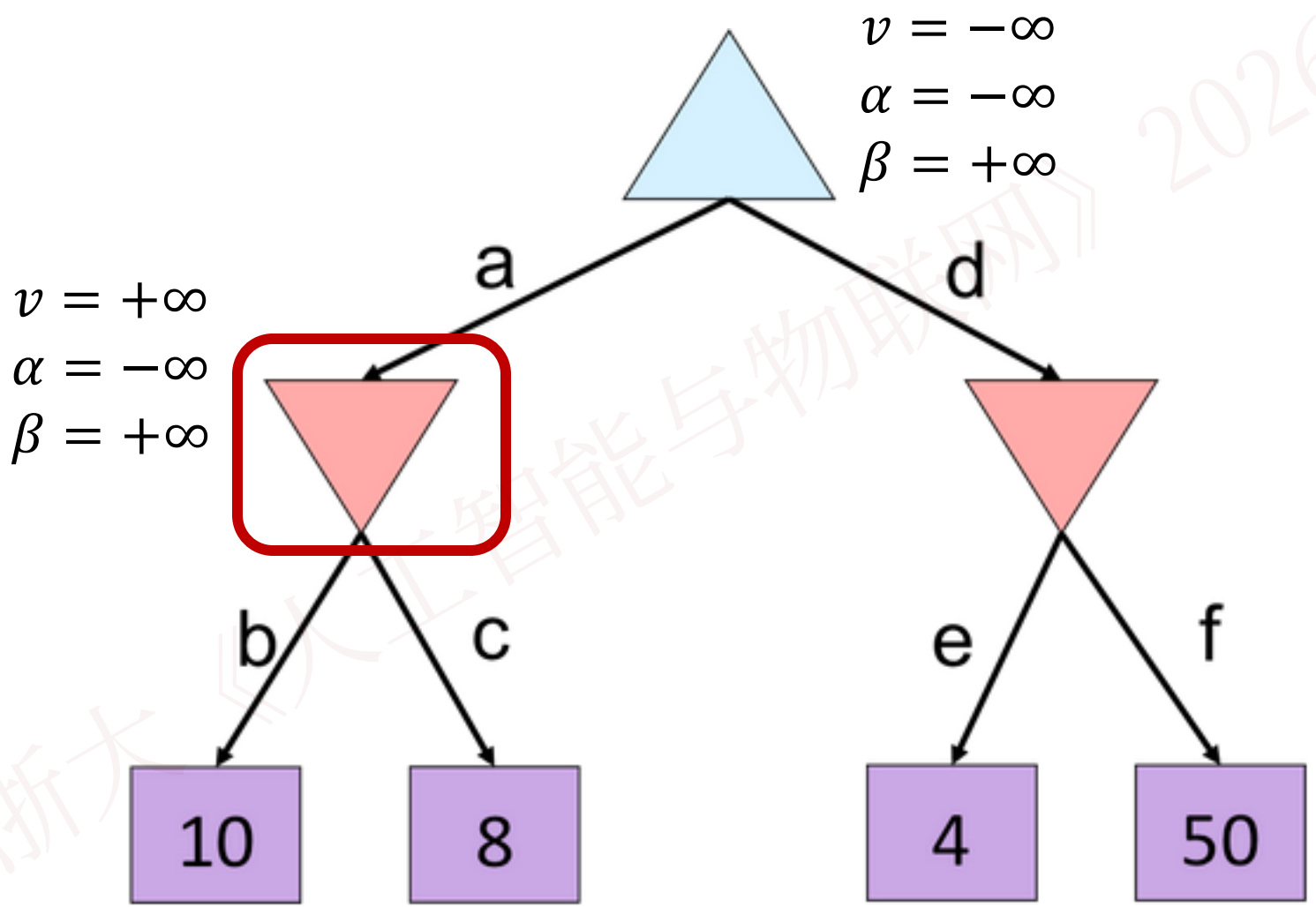
浙大·欧大汇智能与物联网》2026夏

Alpha-Beta Quiz



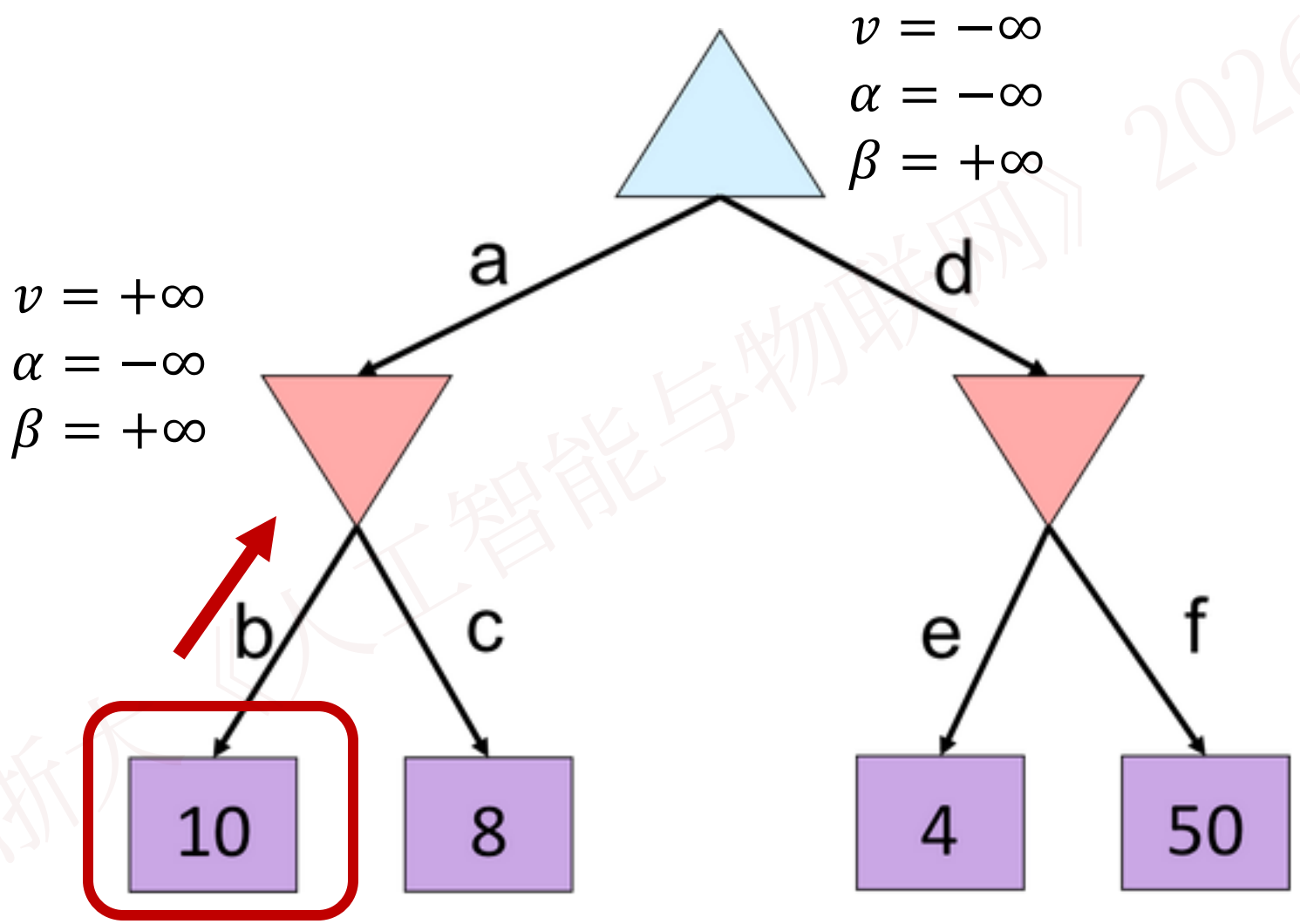
浙大·欧大汇智能与物联网 2026夏

Alpha-Beta Quiz



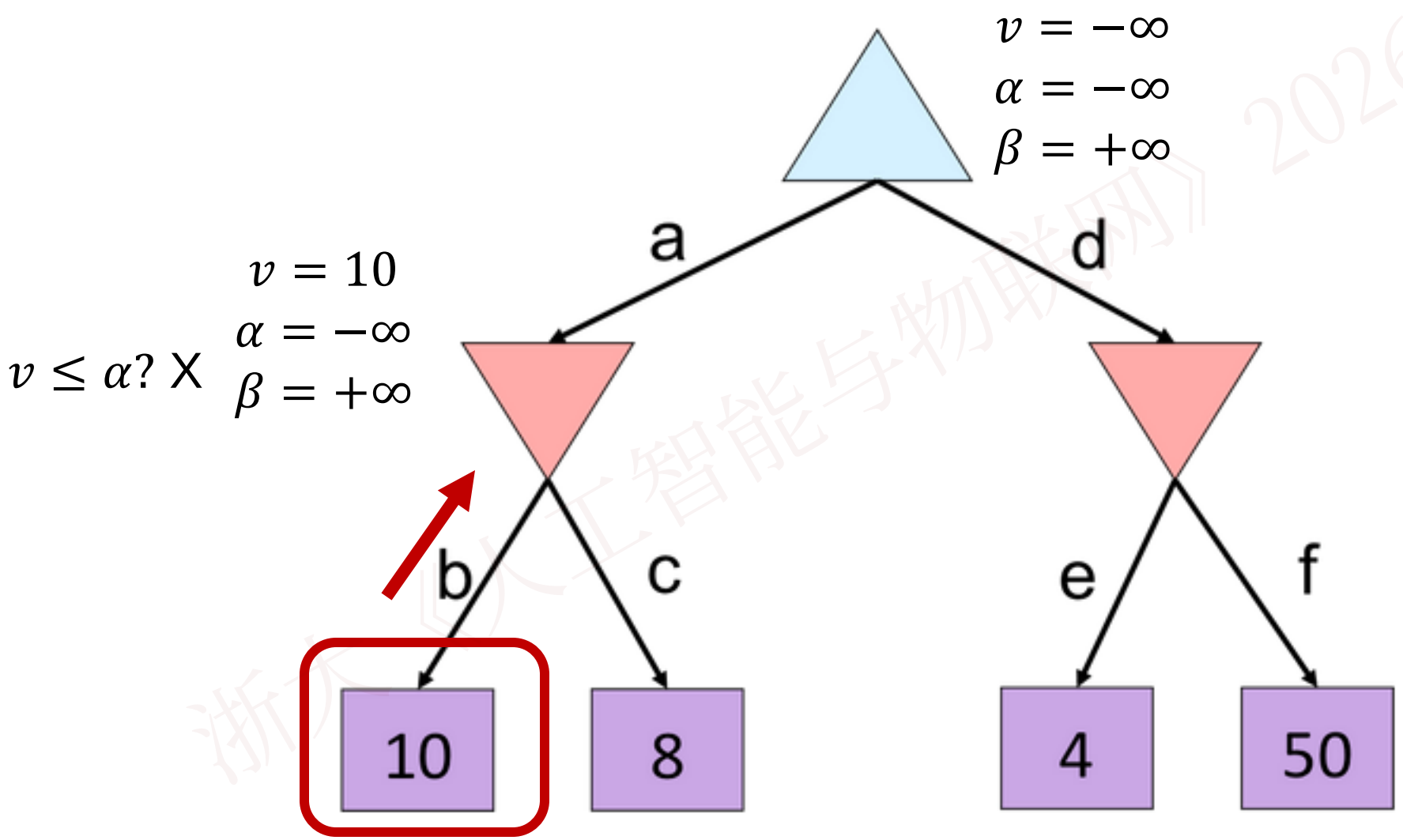
浙大 人工智能与物联网 2026夏

Alpha-Beta Quiz



浙大·迈大汇智能与物联网 2026夏

Alpha-Beta Quiz



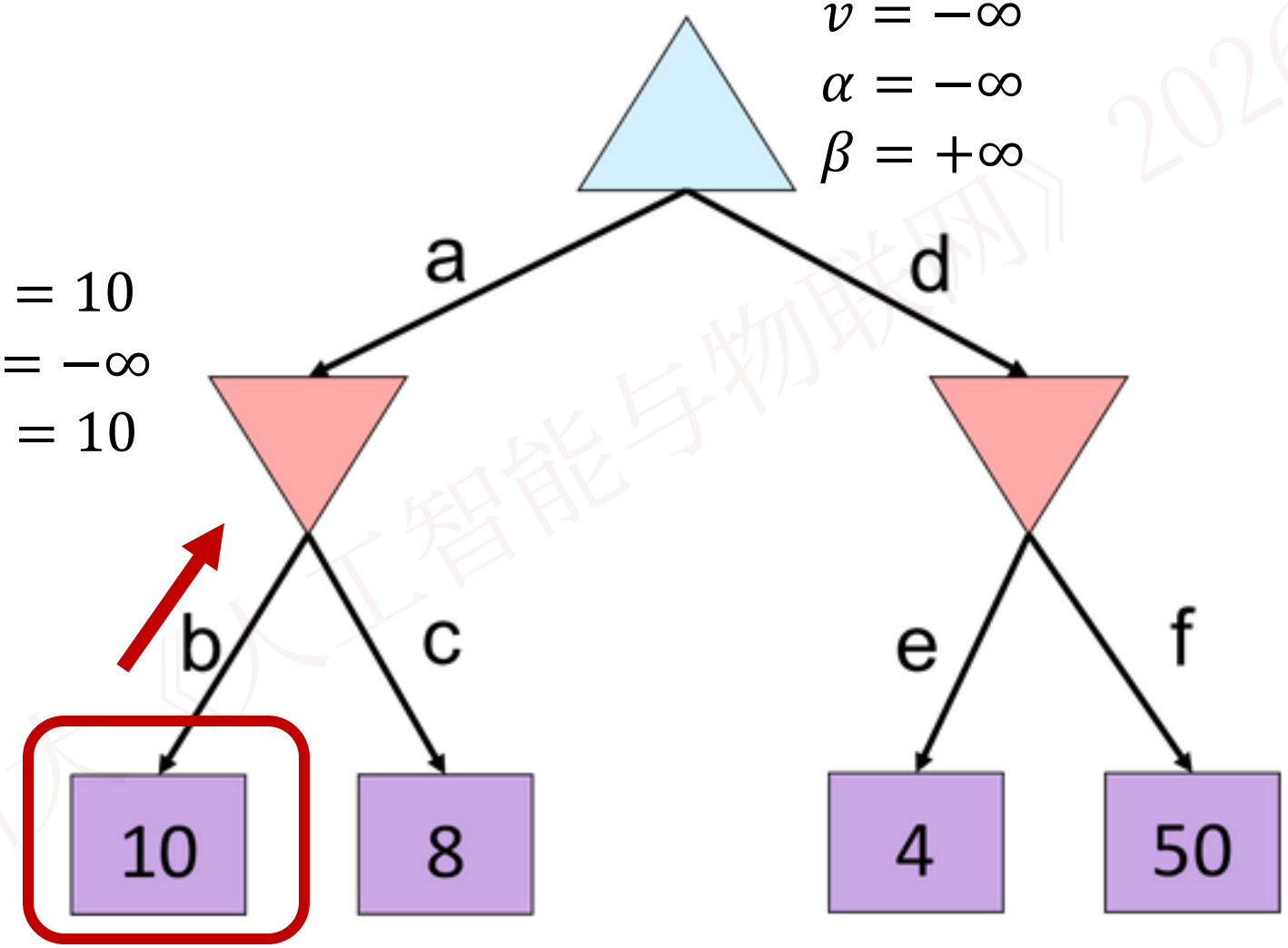
浙大·迈人工智能与物联网 2026夏

Alpha-Beta Quiz

$v = -\infty$
$\alpha = -\infty$
$\beta = +\infty$

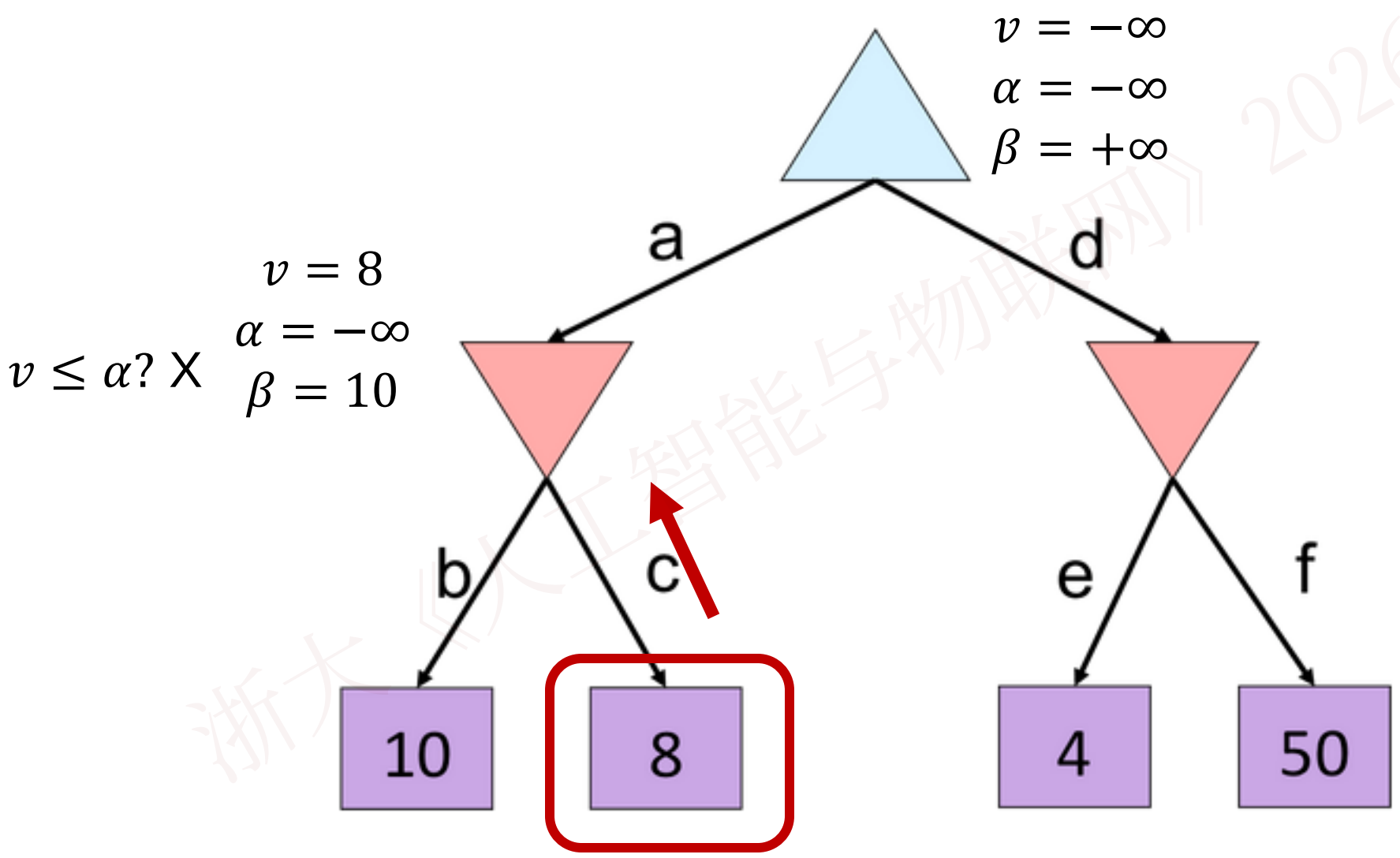
$v = 10$
$\alpha = -\infty$
$\beta = 10$

$\beta = \min(\beta, v)$



浙大·数据汇智能与物联网 2026夏

Alpha-Beta Quiz



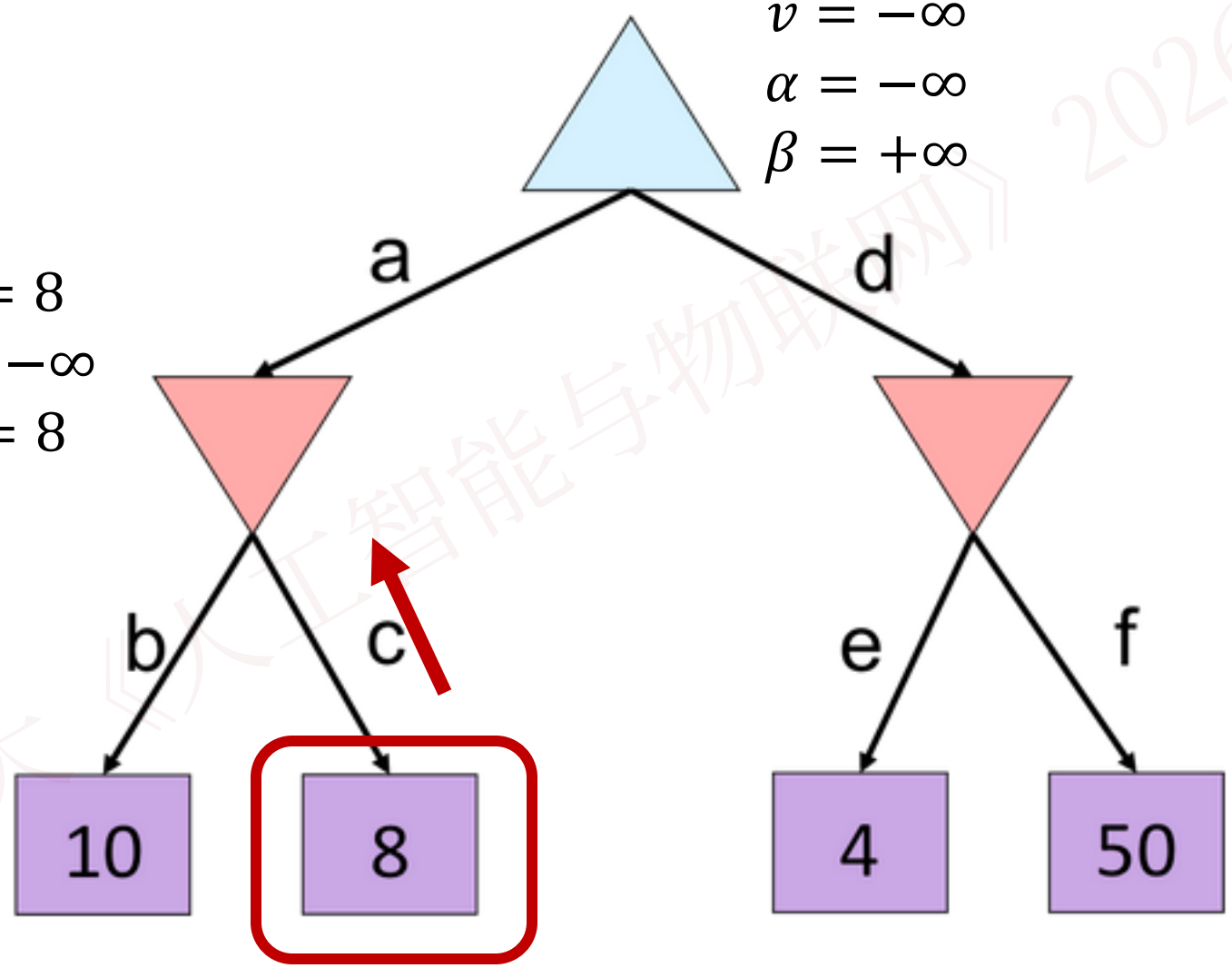
浙大·欧大汇智能与物联网 2026夏

Alpha-Beta Quiz

$v = -\infty$
$\alpha = -\infty$
$\beta = +\infty$

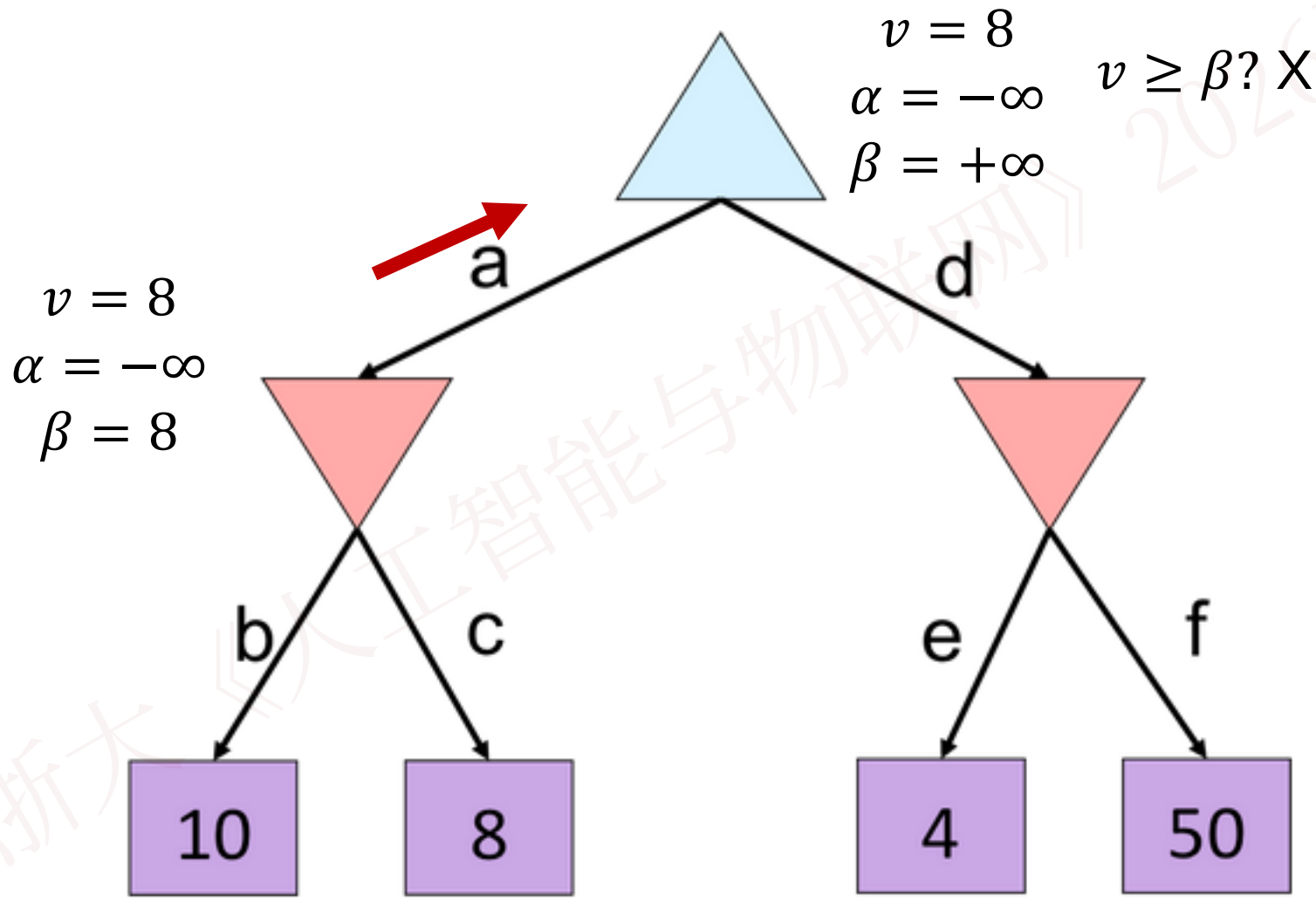
$v = 8$
$\alpha = -\infty$
$\beta = 8$

$\beta = \min(\beta, v)$



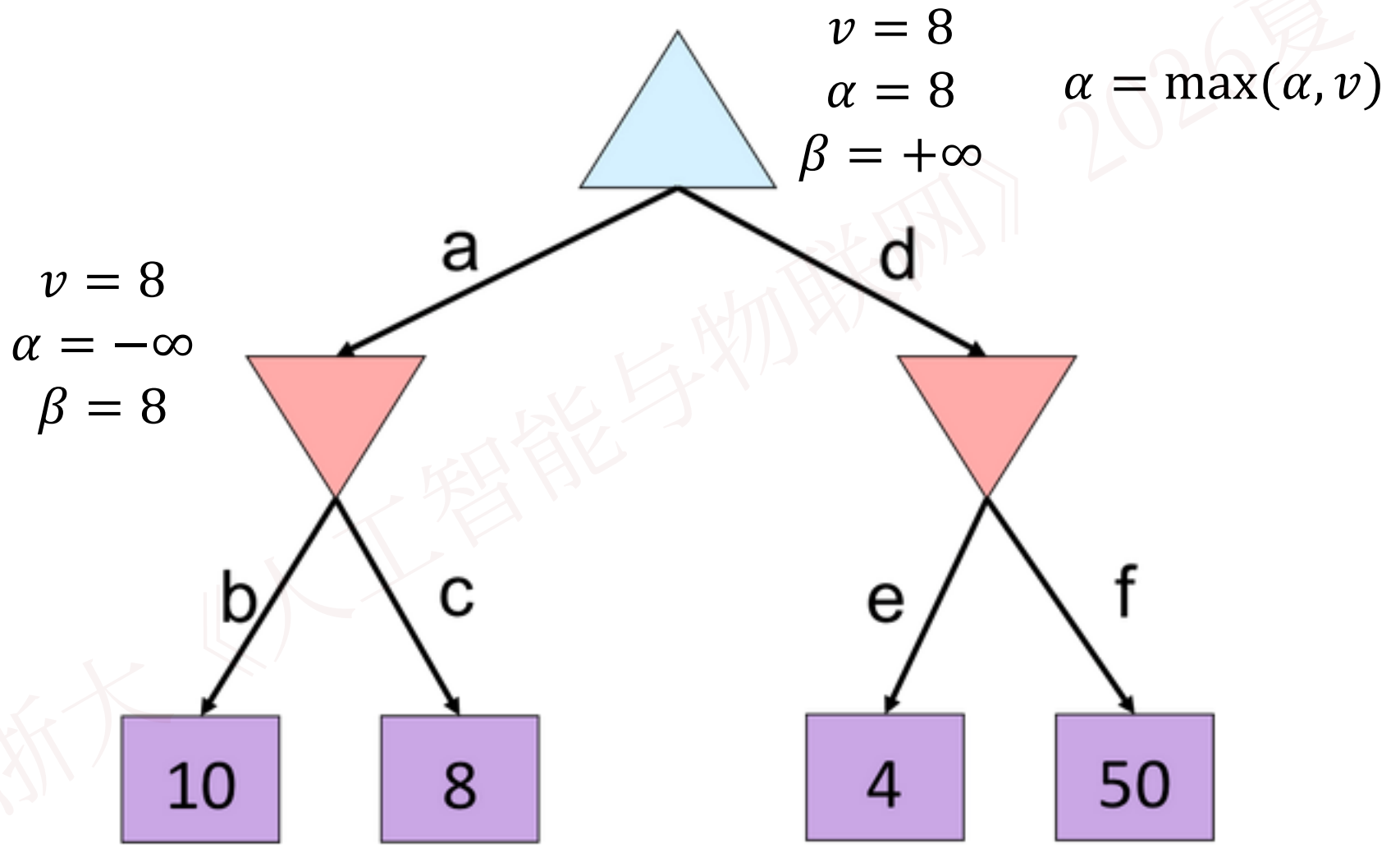
浙大·欧大汇智能与物联网 2026夏

Alpha-Beta Quiz

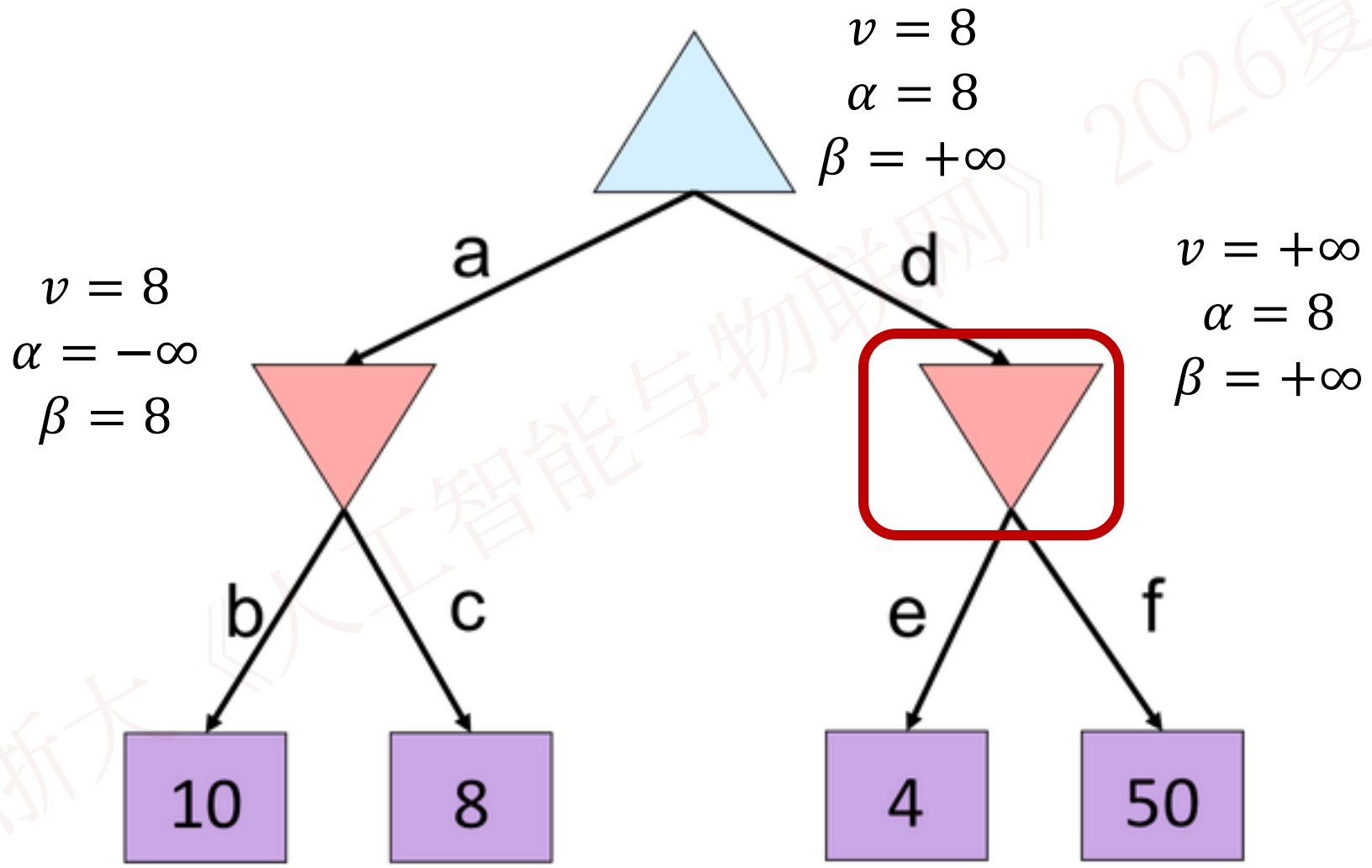


浙大 人工智能与物联网 2020夏

Alpha-Beta Quiz

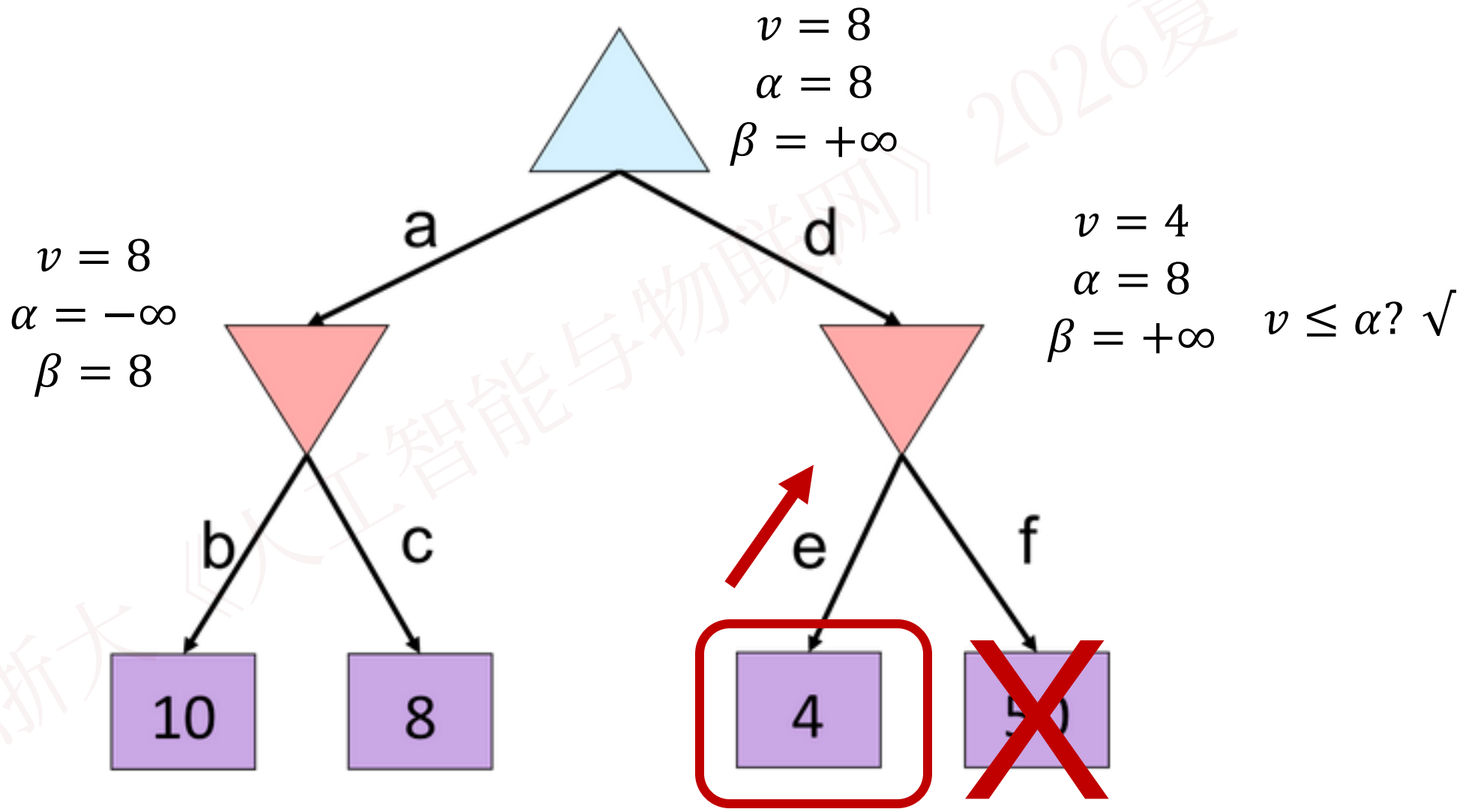


Alpha-Beta Quiz



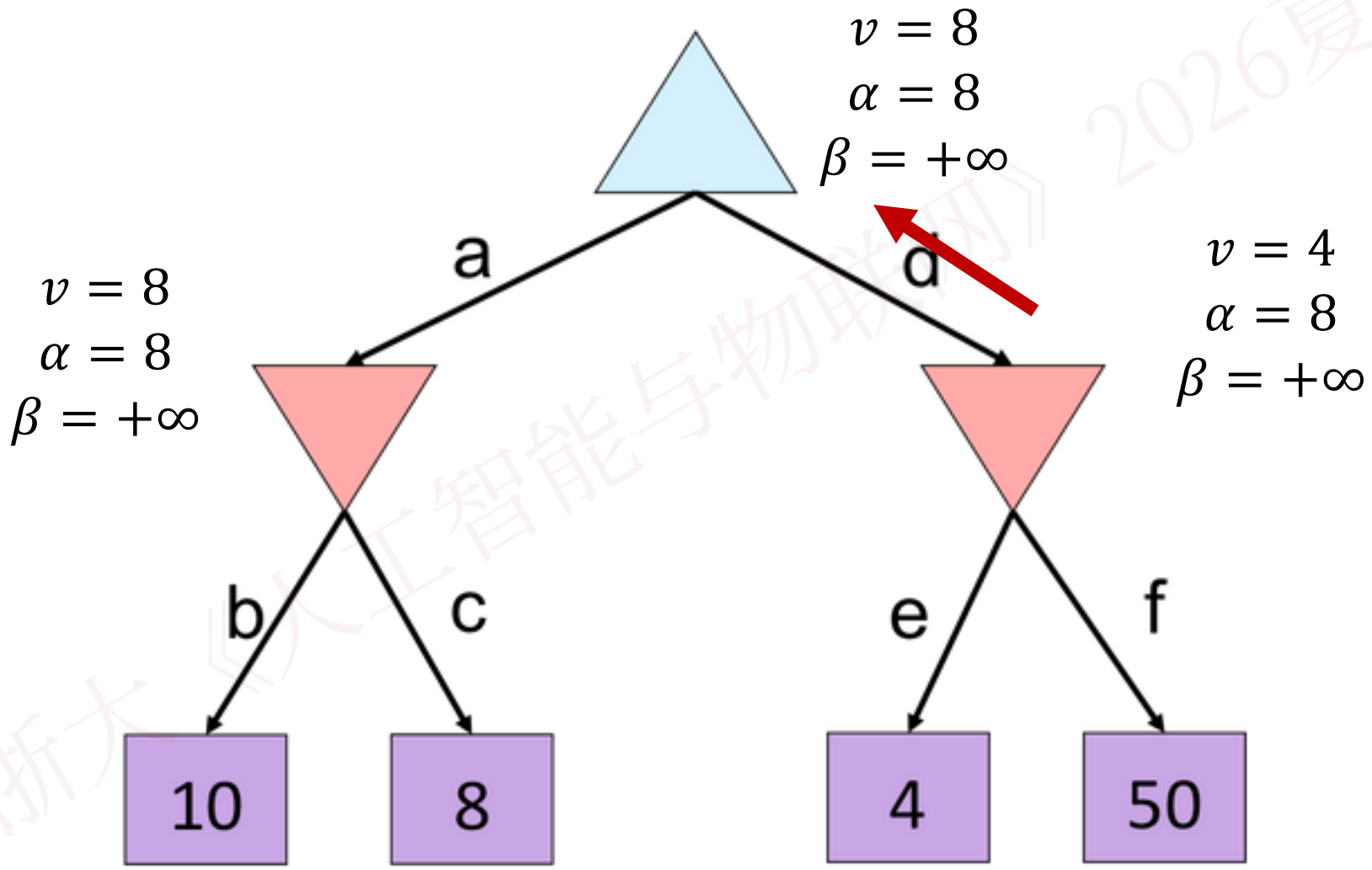
浙大·人工智能与物联网 2026夏

Alpha-Beta Quiz



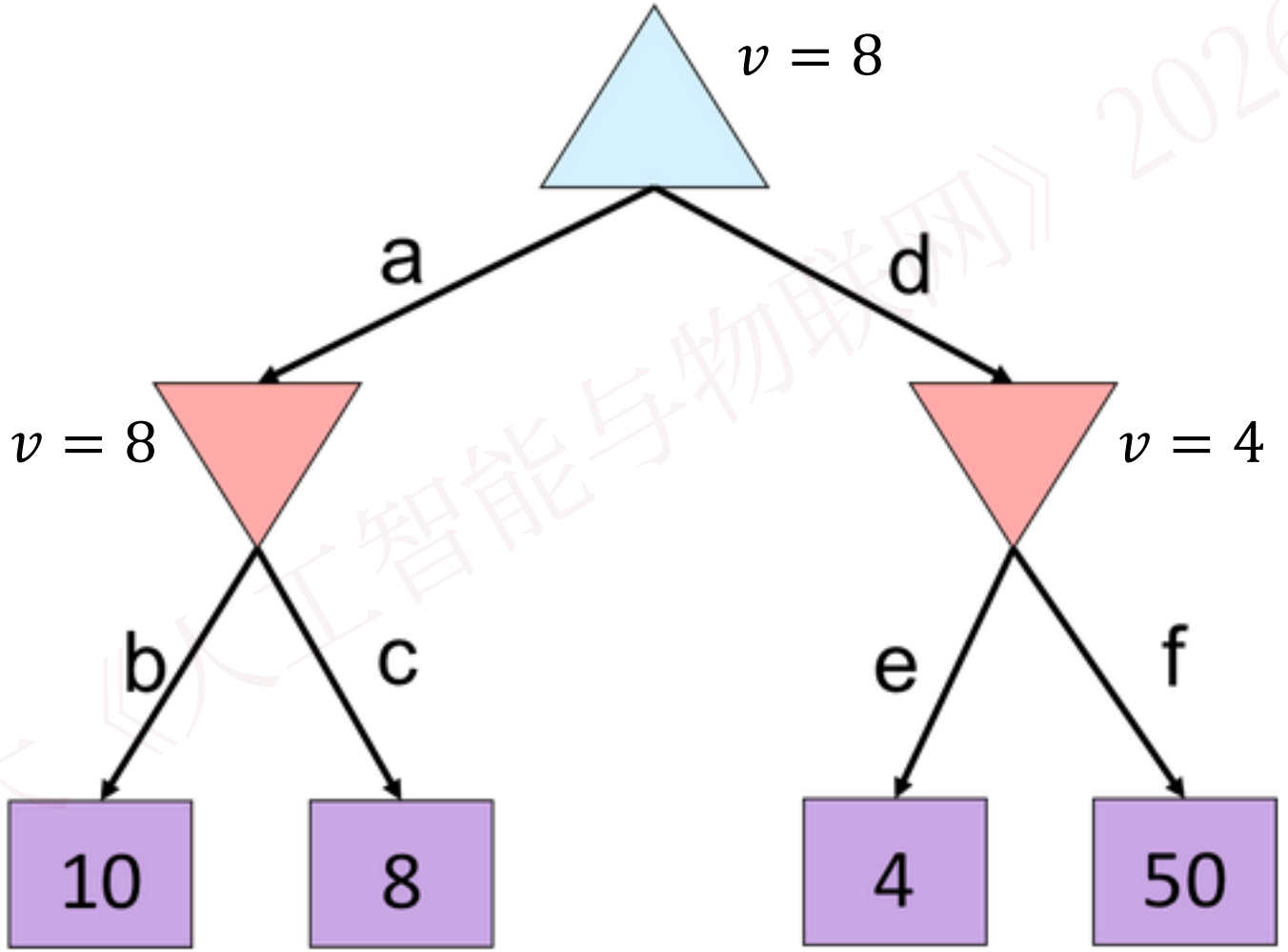
浙大·欧大汇智能与物联网 2026夏

Alpha-Beta Quiz



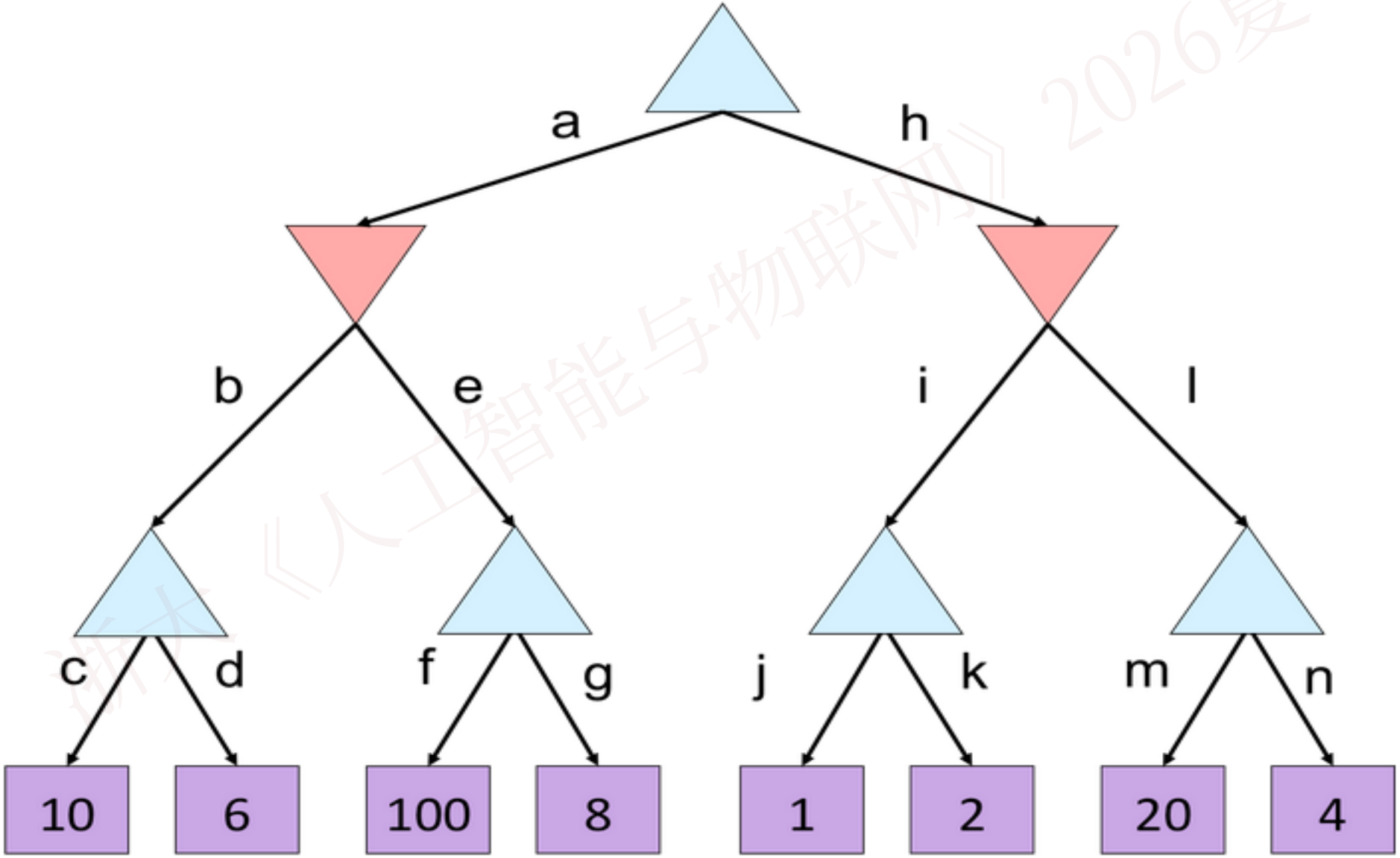
浙大人机交互智能与物联网 2026夏

Alpha-Beta Quiz



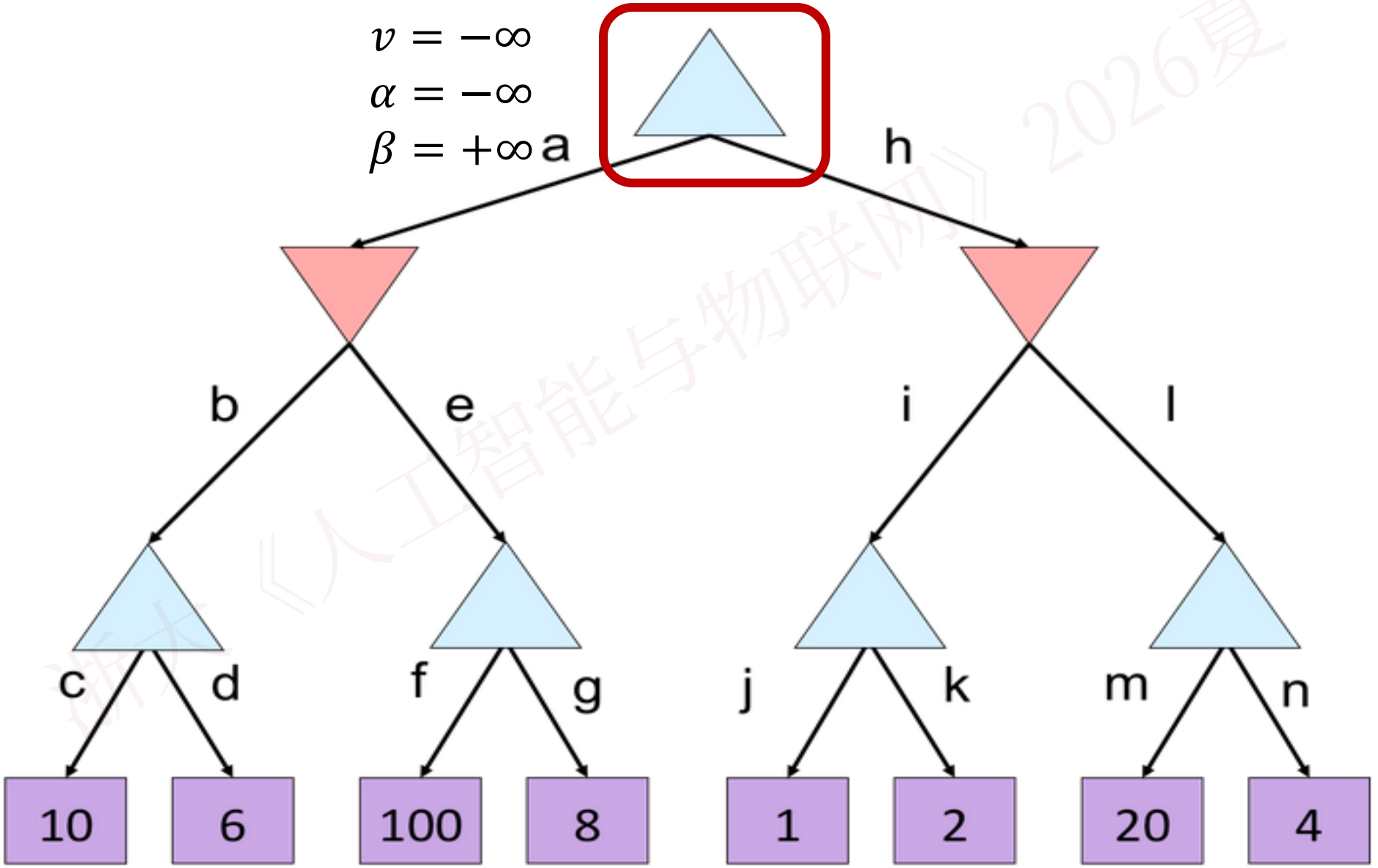
浙大·欧大汇智能与物联网》2026夏

Alpha-Beta Quiz 2

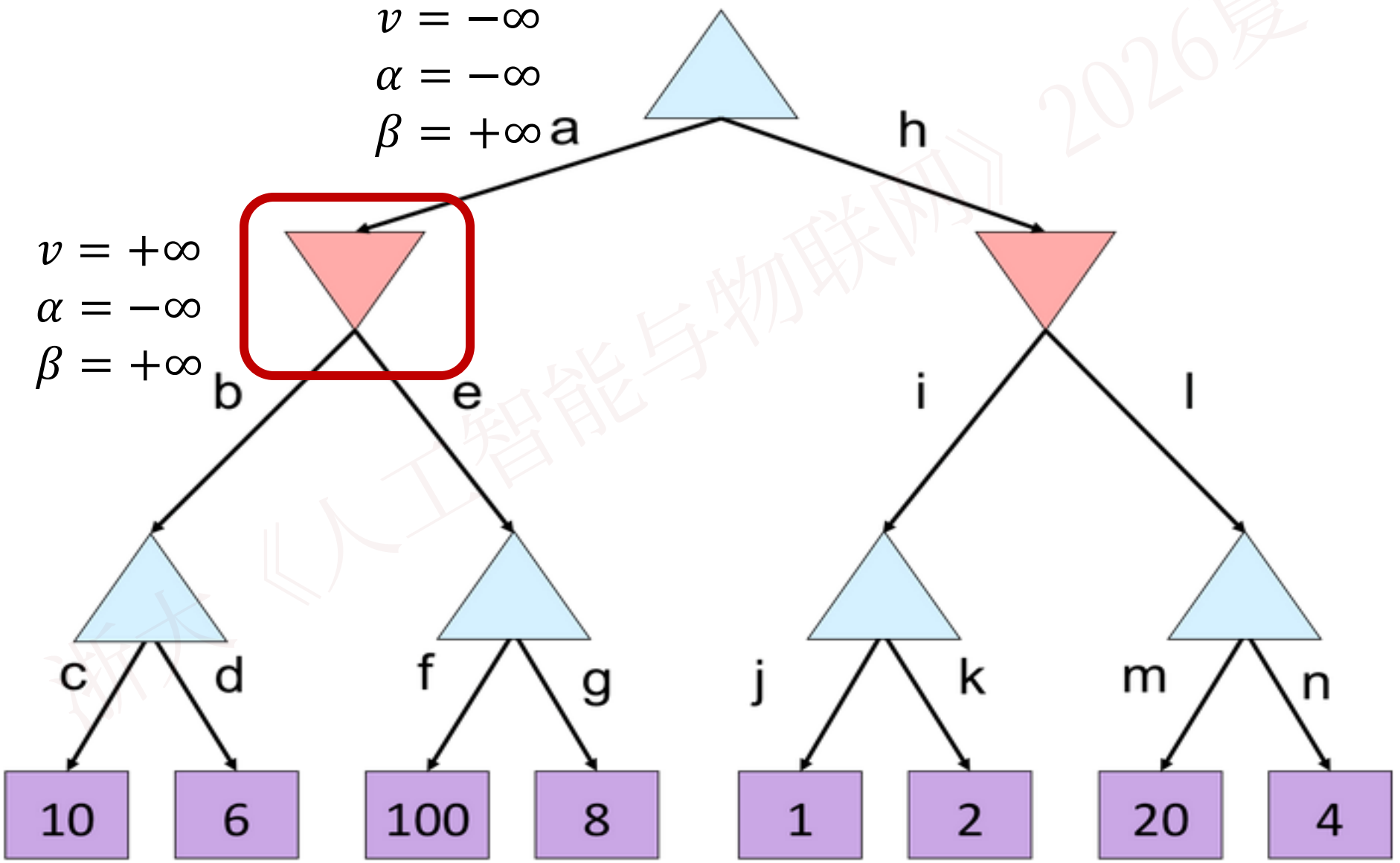


浙大《人工智能与物联网》 2026夏

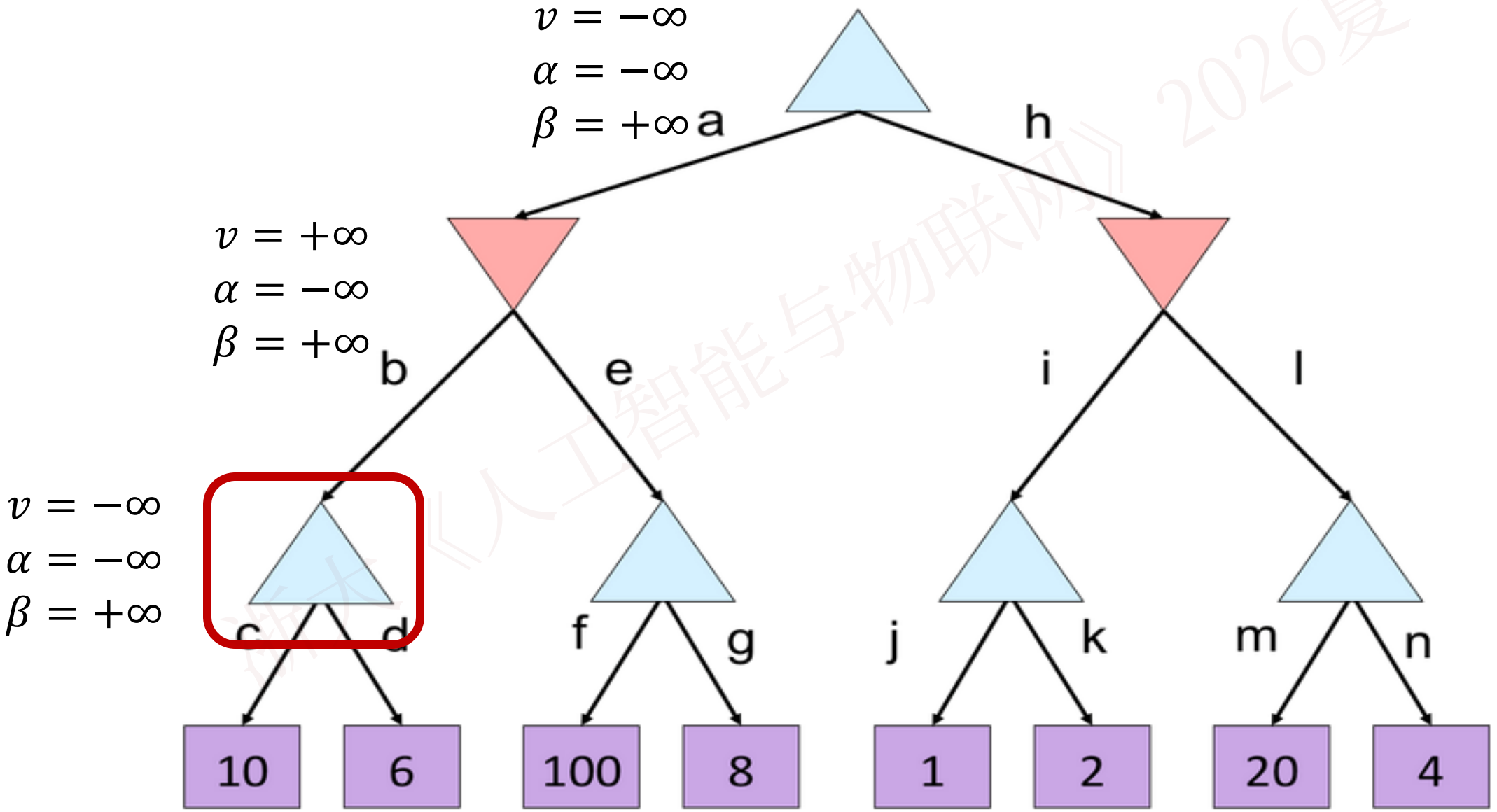
Alpha-Beta Quiz 2



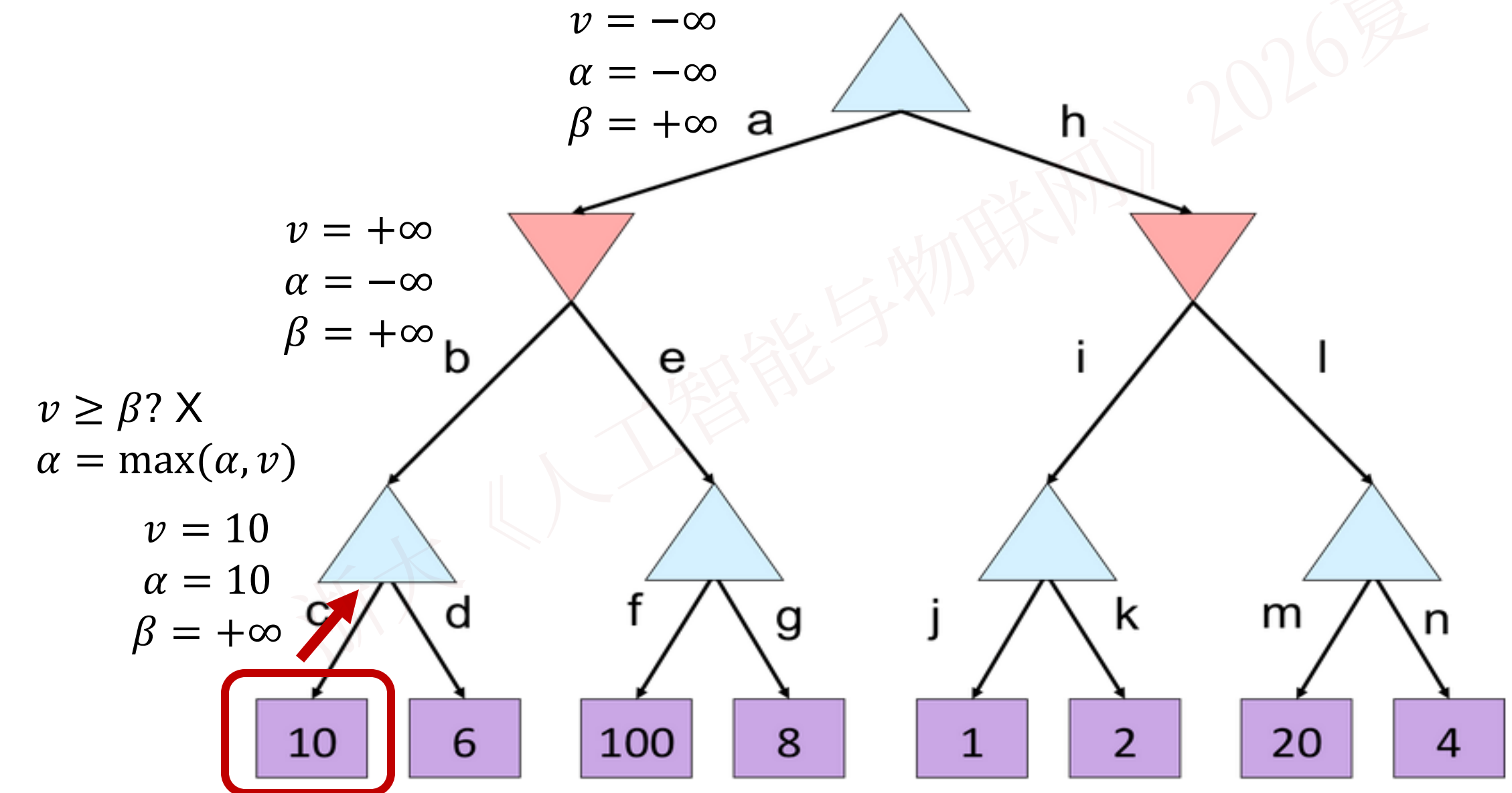
Alpha-Beta Quiz 2



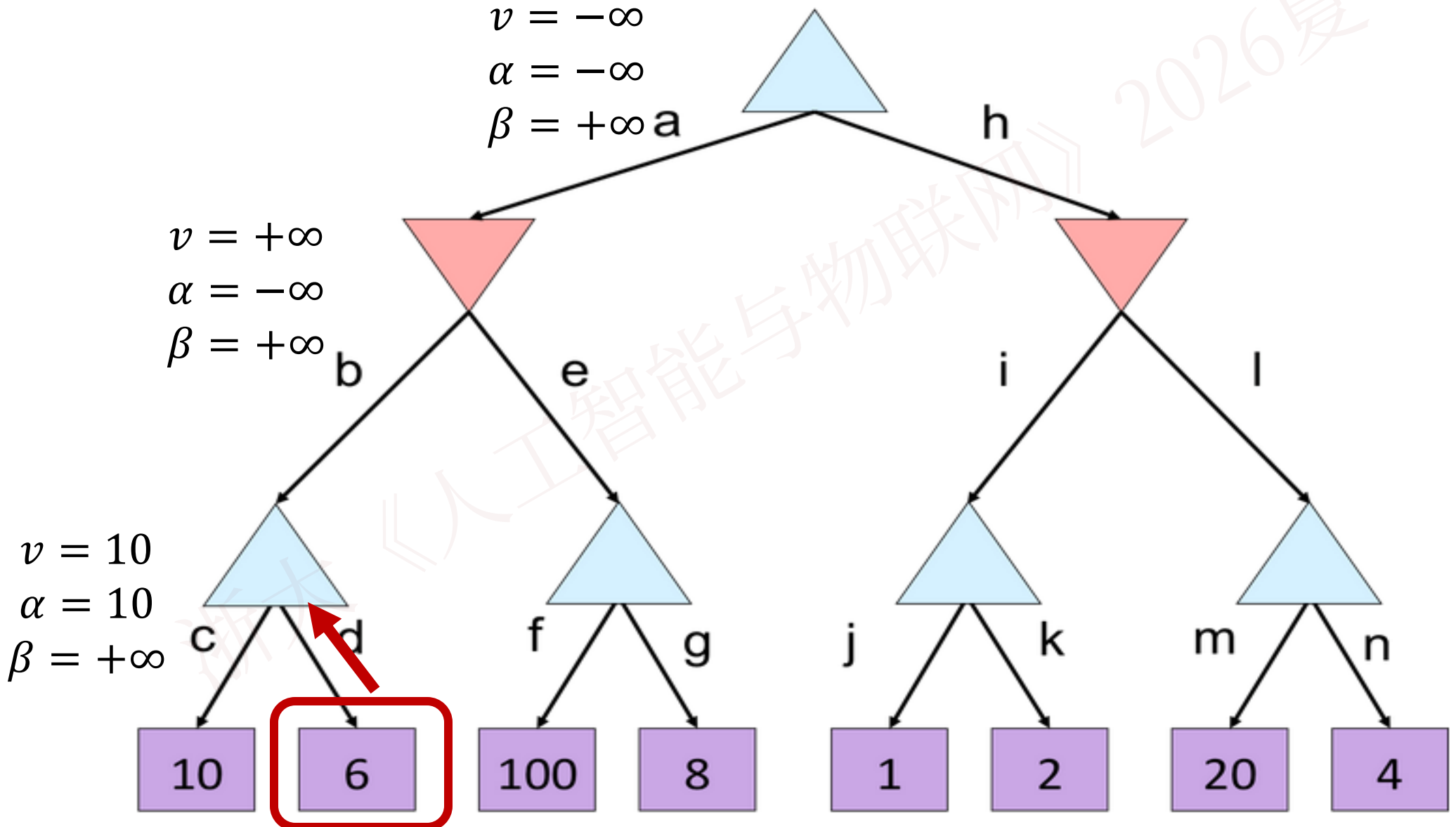
Alpha-Beta Quiz 2



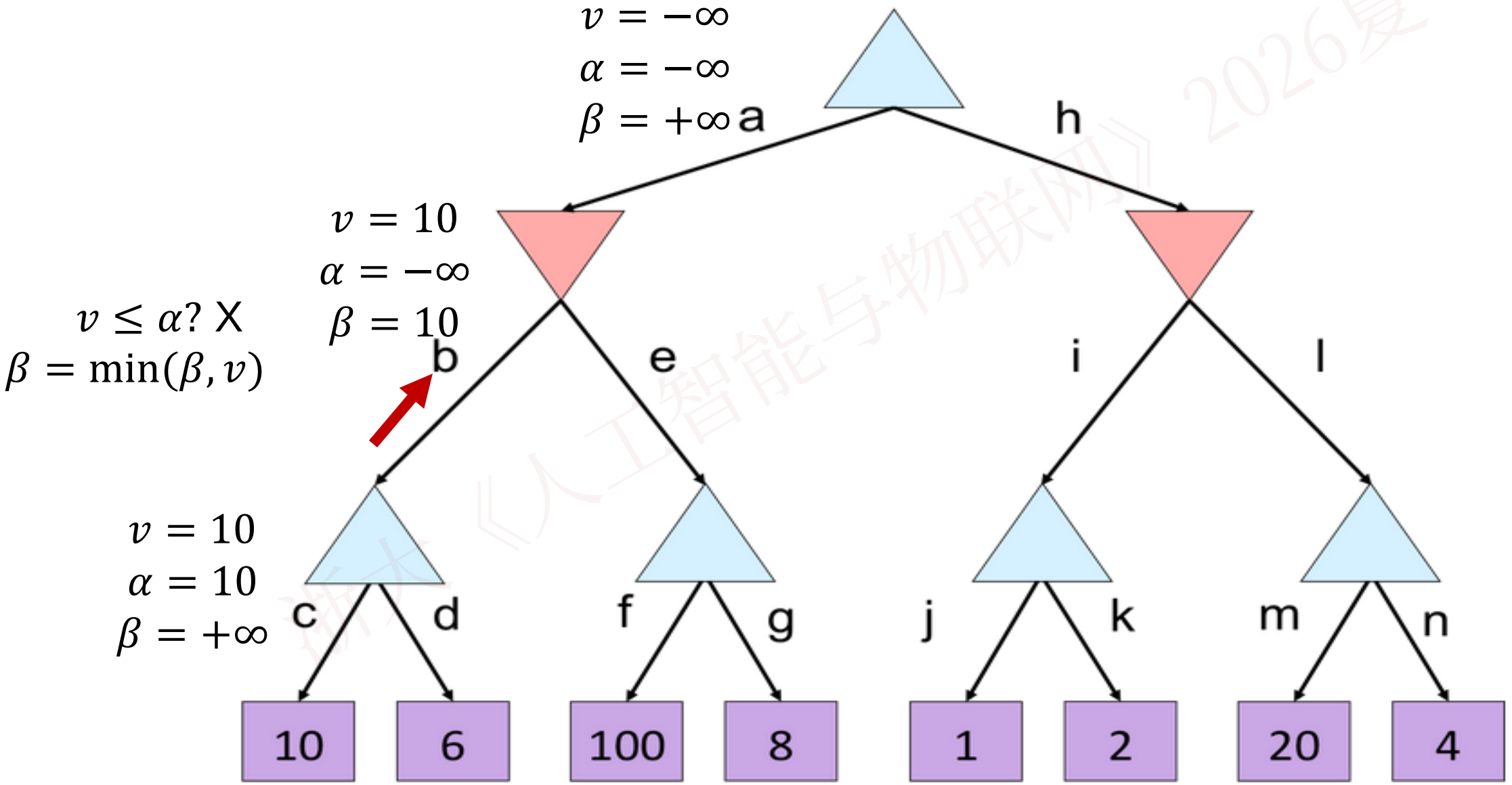
Alpha-Beta Quiz 2



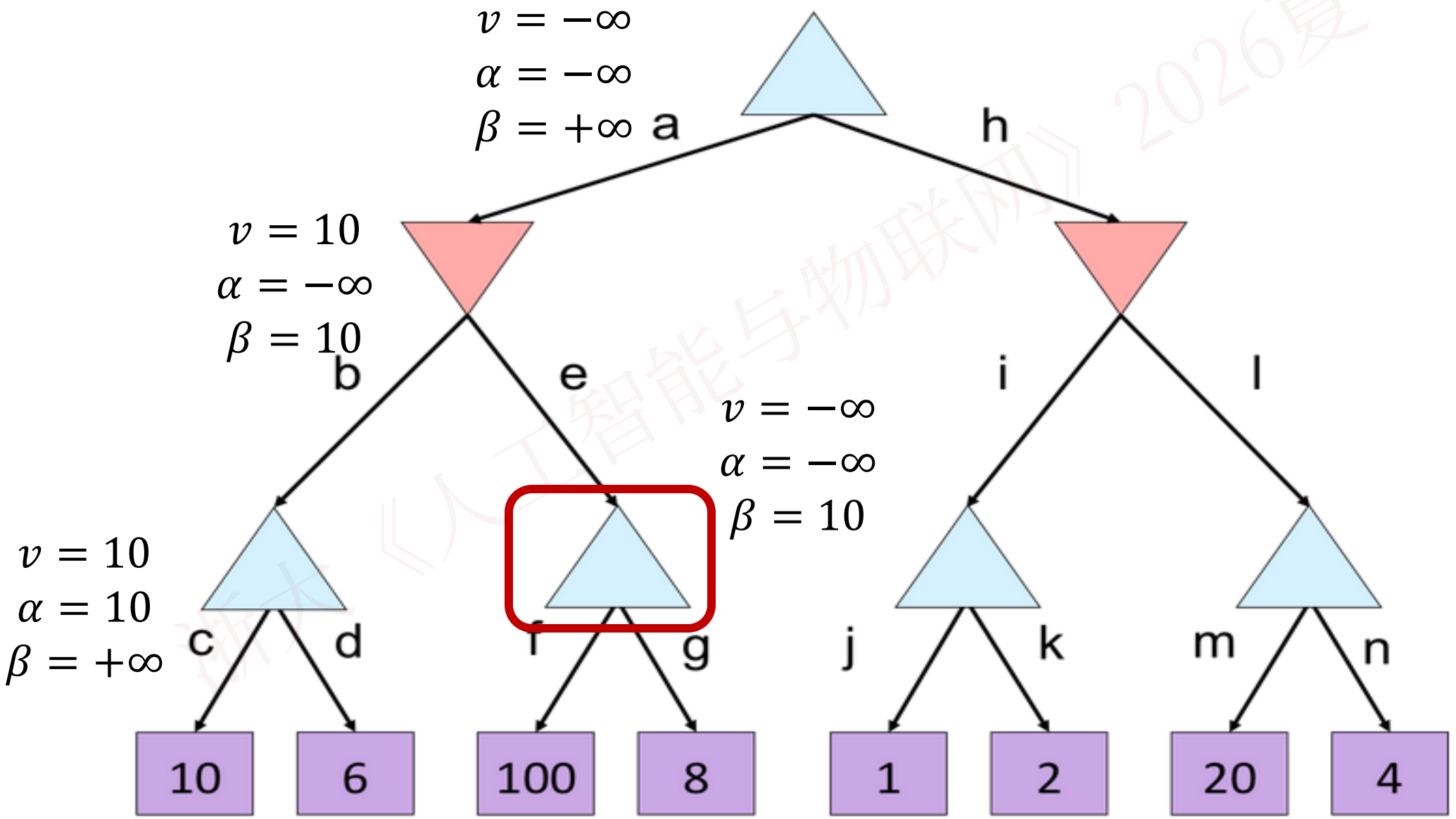
Alpha-Beta Quiz 2



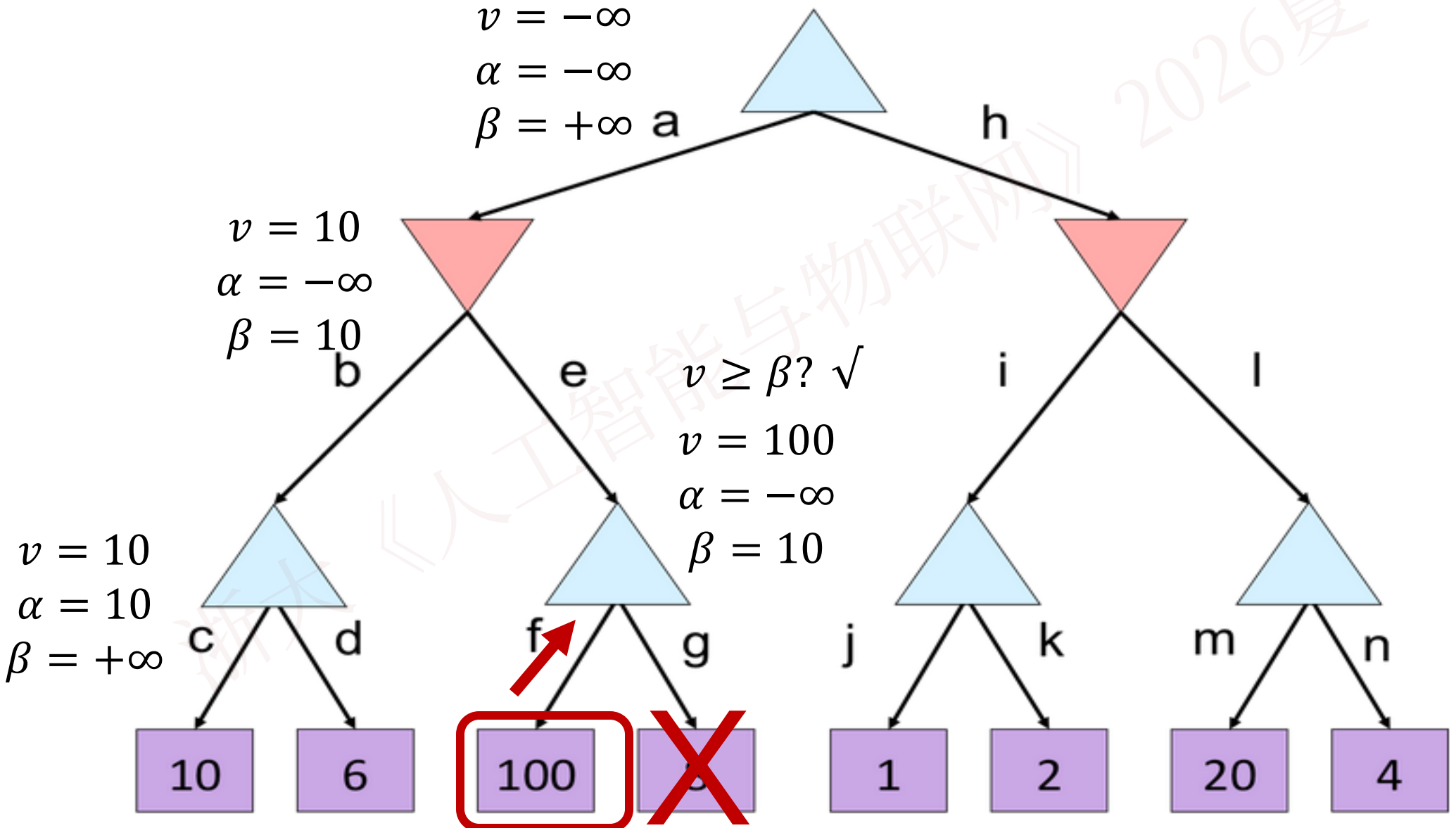
Alpha-Beta Quiz 2



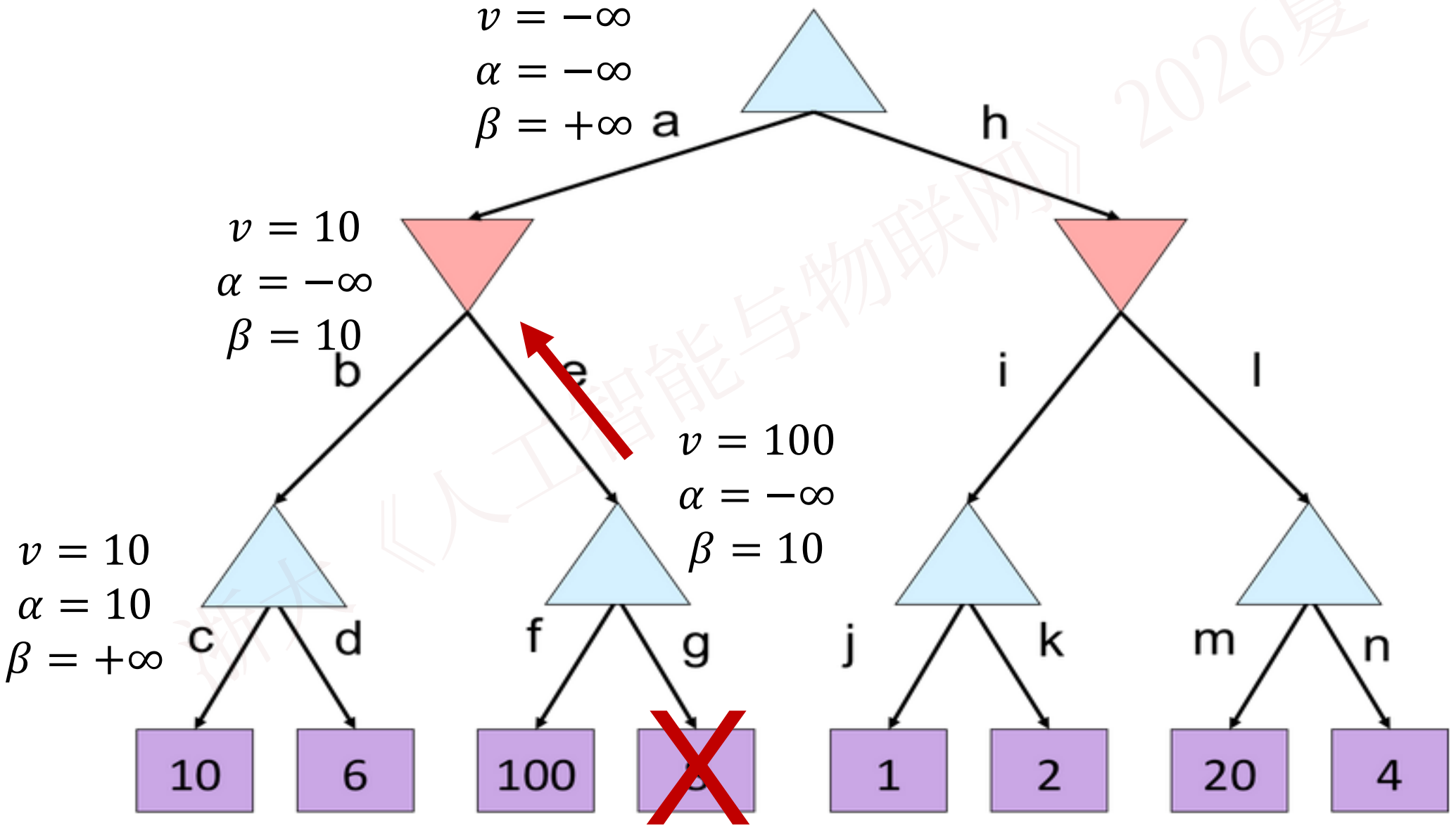
Alpha-Beta Quiz 2



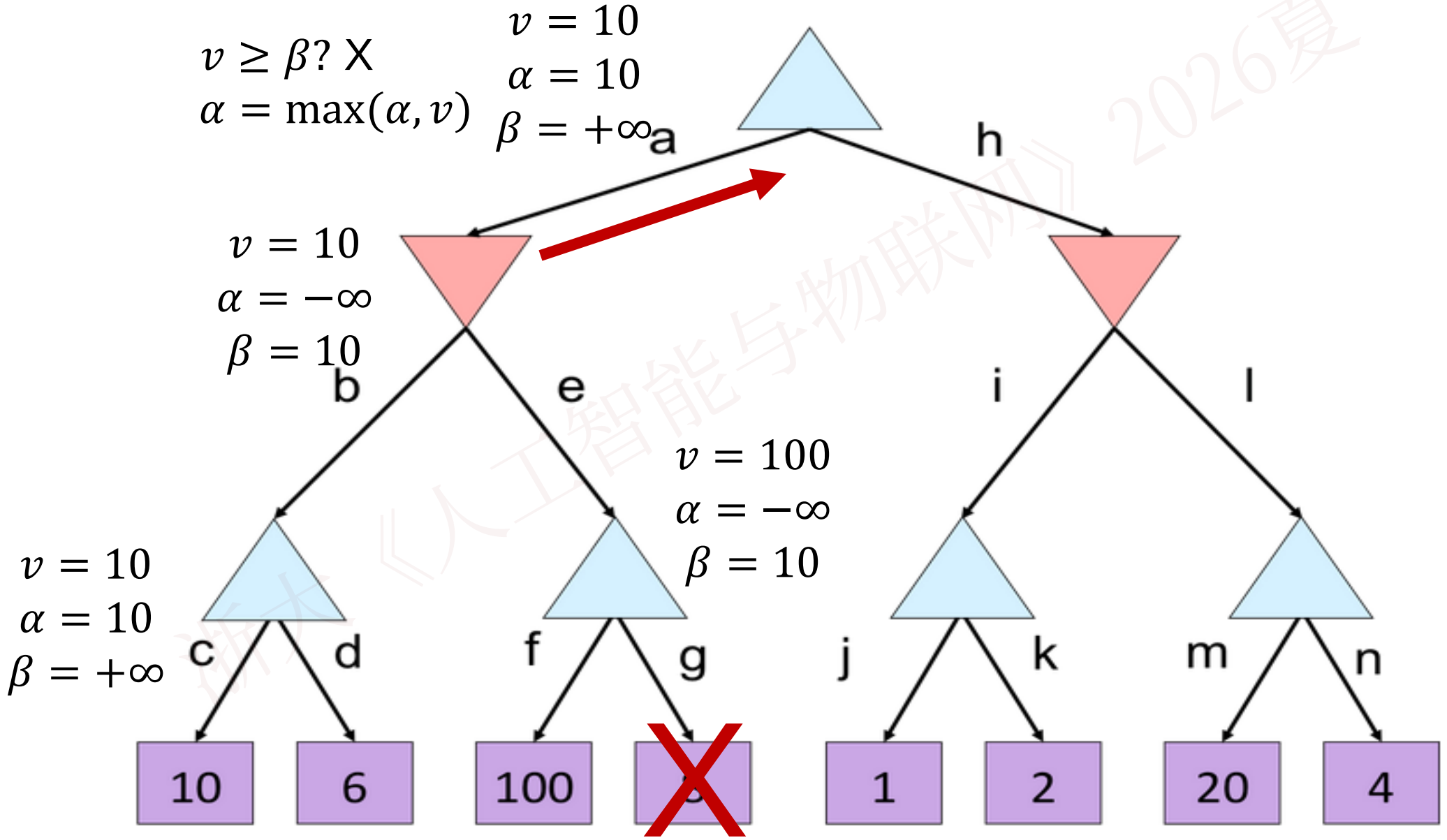
Alpha-Beta Quiz 2



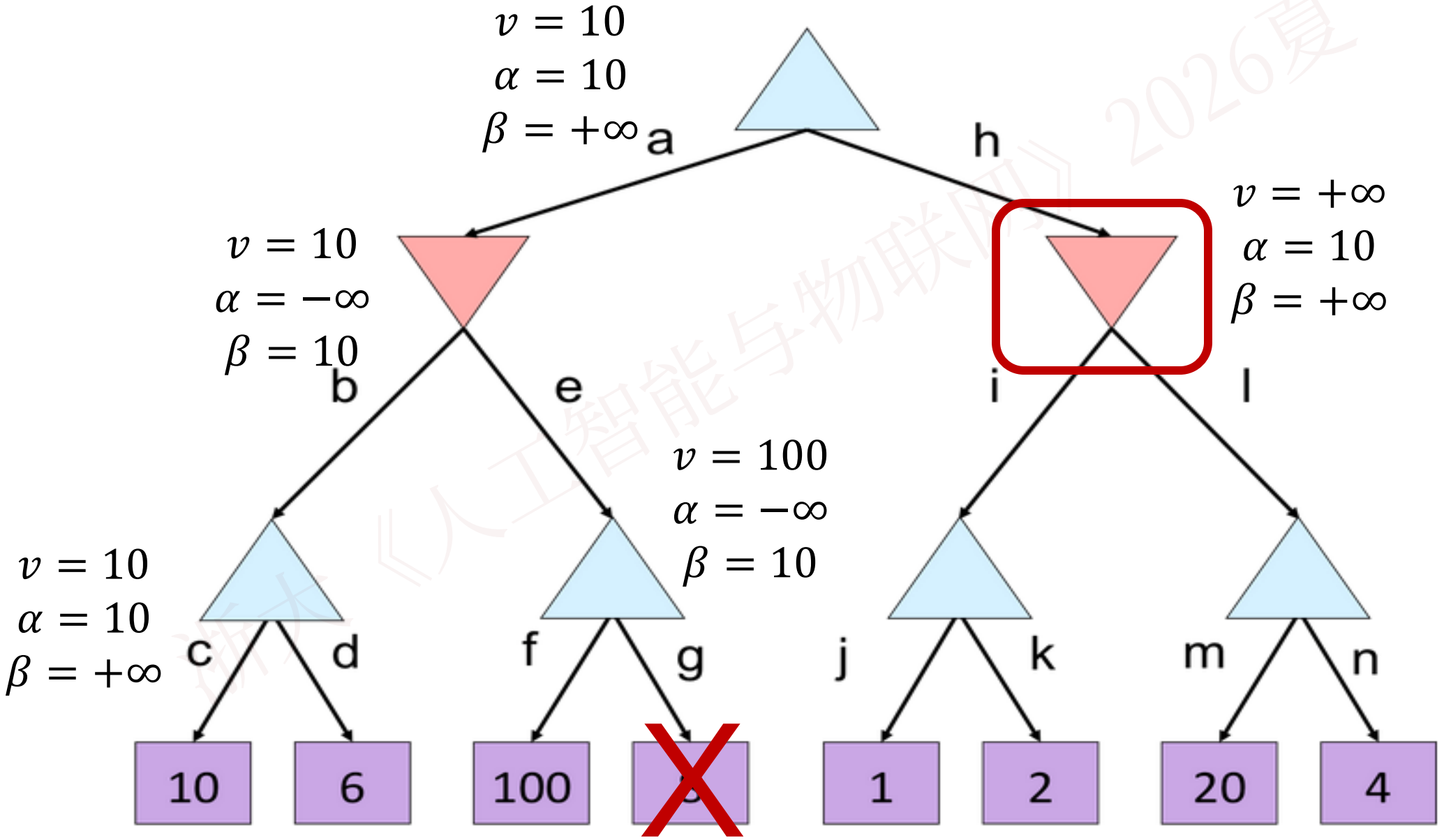
Alpha-Beta Quiz 2



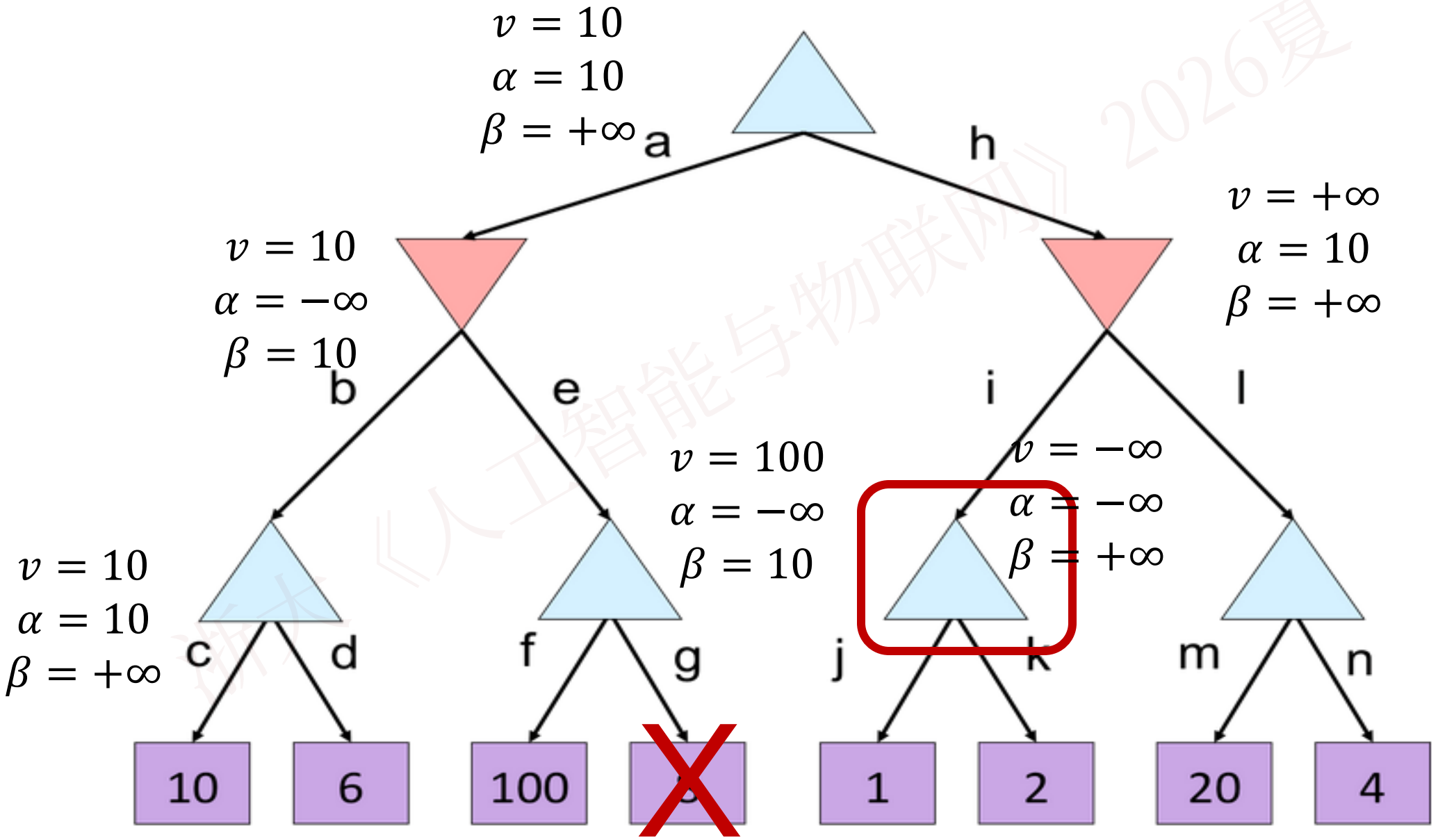
Alpha-Beta Quiz 2



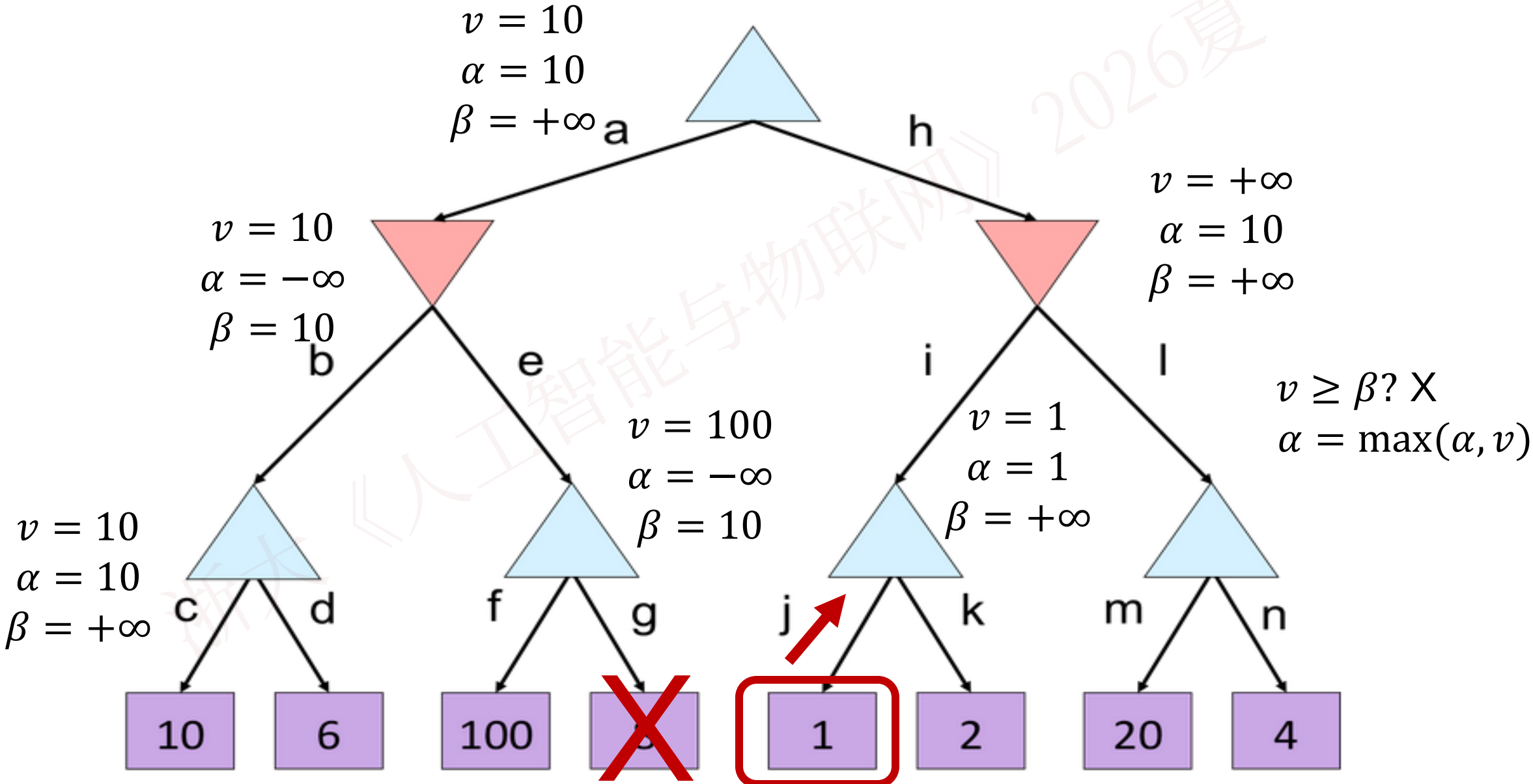
Alpha-Beta Quiz 2



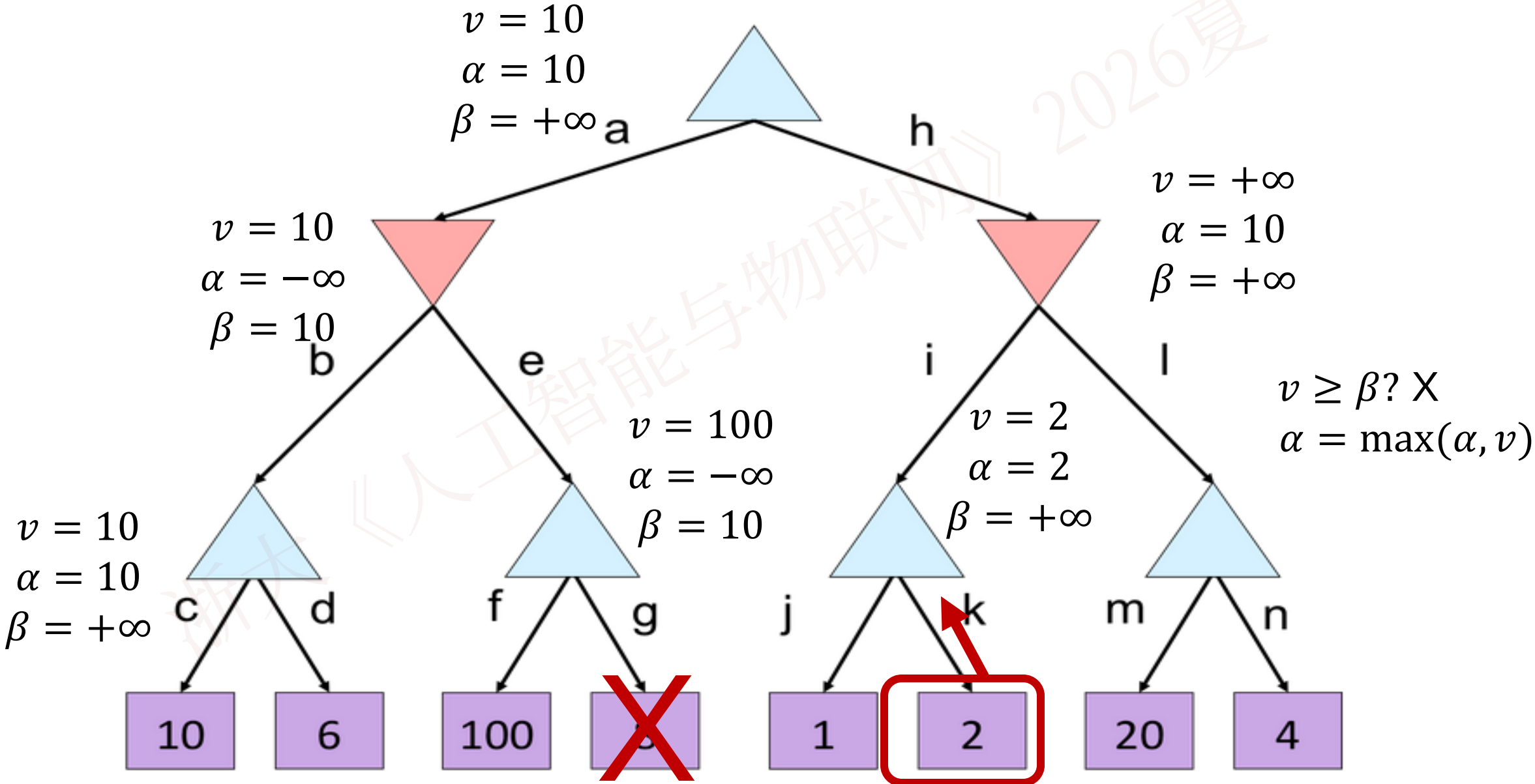
Alpha-Beta Quiz 2



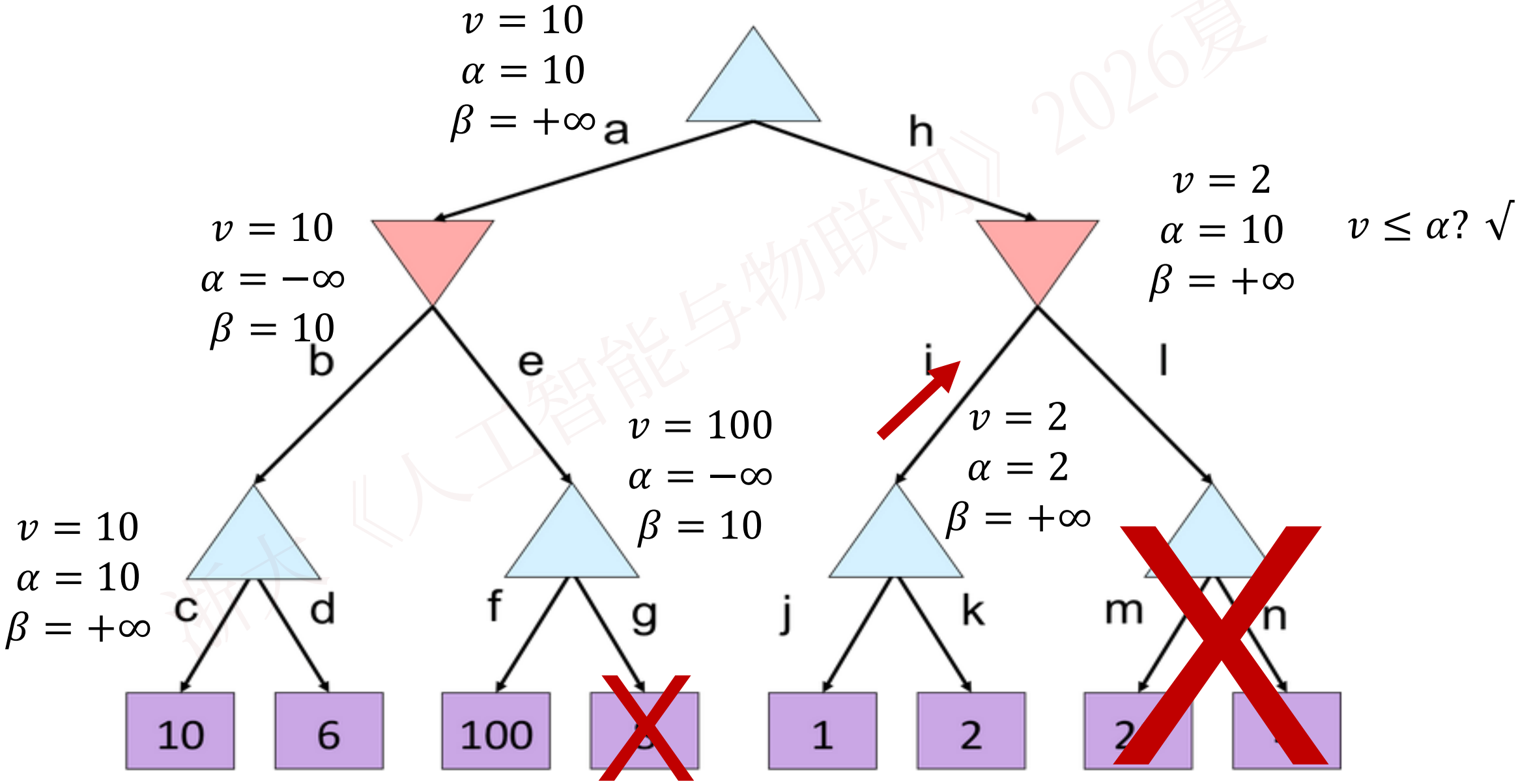
Alpha-Beta Quiz 2



Alpha-Beta Quiz 2



Alpha-Beta Quiz 2



Alpha-Beta Quiz 2

